
Python and HDF5

Andrew Collette

Introduction

When I was a graduate student, I had a serious problem: a brand-new dataset, made up of millions of data points collected painstakingly over a full week on a nationally recognized plasma research device, that contained values that were much too small.

About *40 orders of magnitude* too small.

My advisor and I huddled in his office, in front of the shiny new G5 Power Mac that ran our visualization suite, and tried to figure out what was wrong. The data had been acquired correctly from the machine. It looked like the original raw file from the experiment's digitizer was fine. I had written a (very large) script in the IDL programming language on my Thinkpad laptop to turn the raw data into files the visualization tool could use. This in-house format was simplicity itself: just a short fixed-width header and then a binary dump of the floating-point data. Even so, I spent another hour or so writing a program to verify and plot the files on my laptop. They were fine. And yet, when loaded into the visualizer, all the data that looked so beautiful in IDL turned into a featureless, unstructured mush of values all around 10^{-41} .

Finally it came to us: both the digitizer machines and my Thinkpad used the “little-endian” format to represent floating-point numbers, in contrast to the “big-endian” format of the G5 Mac. Raw values written on one machine couldn't be read on the other, and vice versa. I remember thinking *that's so stupid* (among other less polite variations). Learning that this problem was so common that IDL supplied a special routine to deal with it (SWAP_ENDIAN) did not improve my mood.

At the time, I didn't care that much about the details of how my data was stored. This incident and others like it changed my mind. As a scientist, I eventually came to recognize that the choices we make for organizing and storing our data are also choices about communication. Not only do standard, well-designed formats make life easier for individuals (and eliminate silly time-wasters like the “endian” problem), but they make it possible to share data with a global audience.

Python and HDF5

In the Python world, consensus is rapidly converging on Hierarchical Data Format version 5, or “HDF5,” as the standard mechanism for storing large quantities of numerical data. As data volumes get larger, organization of data becomes increasingly important; features in HDF5 like named datasets ([Chapter 3](#)), hierarchically organized groups ([Chapter 5](#)), and user-defined metadata “attributes” ([Chapter 6](#)) become essential to the analysis process.

Structured, “self-describing” formats like HDF5 are a natural complement to Python. Two production-ready, feature-rich interface packages exist for HDF5, h5py, and PyTables, along with a number of smaller special-purpose wrappers.

Organizing Data and Metadata

Here’s a simple example of how HDF5’s structuring capability can help an application. Don’t worry too much about the details; later chapters explain both the details of how the file is structured, and how to use the HDF5 API from Python. Consider this a taste of what HDF5 can do for your application. If you want to follow along, you’ll need Python 2 with NumPy installed (see [Chapter 2](#)).

Suppose we have a NumPy array that represents some data from an experiment:

```
>>> import numpy as np
>>> temperature = np.random.random(1024)
>>> temperature
array([ 0.44149738,  0.7407523 ,  0.44243584, ...,  0.19018119,
        0.64844851,  0.55660748])
```

Let’s also imagine that these data points were recorded from a weather station that sampled the temperature, say, every 10 seconds. In order to make sense of the data, we have to record that sampling interval, or “delta-T,” somewhere. For now we’ll put it in a Python variable:

```
>>> dt = 10.0
```

The data acquisition started at a particular time, which we will also need to record. And of course, we have to know that the data came from Weather Station 15:

```
>>> start_time = 1375204299 # in Unix time
>>> station = 15
```

We could use the built-in NumPy function `np.savez` to store these values on disk. This simple function saves the values as NumPy arrays, packed together in a ZIP file with associated names:

```
>>> np.savez("weather.npz", data=temperature, start_time=start_time, station=
station)
```

We can get the values back from the file with `np.load`:

```
>>> out = np.load("weather.npz")
>>> out["data"]
array([ 0.44149738,  0.7407523 ,  0.44243584, ...,  0.19018119,
        0.64844851,  0.55660748])
>>> out["start_time"]
array(1375204299)
>>> out["station"]
array(15)
```

So far so good. But what if we have more than one quantity per station? Say there's also wind speed data to record?

```
>>> wind = np.random.random(2048)
>>> dt_wind = 5.0 # Wind sampled every 5 seconds
```

And suppose we have multiple stations. We could introduce some kind of naming convention, I suppose: “wind_15” for the wind values from station 15, and things like “dt_wind_15” for the sampling interval. Or we could use multiple files...

In contrast, here's how this application might approach storage with HDF5:

```
>>> import h5py
>>> f = h5py.File("weather.hdf5")
>>> f["/15/temperature"] = temperature
>>> f["/15/temperature"].attrs["dt"] = 10.0
>>> f["/15/temperature"].attrs["start_time"] = 1375204299
>>> f["/15/wind"] = wind
>>> f["/15/wind"].attrs["dt"] = 5.0
...
>>> f["/20/temperature"] = temperature_from_station_20
...
```

(and so on)

This example illustrates two of the “killer features” of HDF5: organization in hierarchical groups and attributes. Groups, like folders in a filesystem, let you store related datasets together. In this case, temperature and wind measurements from the same weather station are stored together under groups named “/15,” “/20,” etc. Attributes let you attach descriptive metadata *directly to the data it describes*. So if you give this file to a colleague, she can easily discover the information needed to make sense of the data:

```
>>> dataset = f["/15/temperature"]
>>> for key, value in dataset.attrs.iteritems():
...     print "%s: %s" % (key, value)
dt: 10.0
start_time: 1375204299
```

Coping with Large Data Volumes

As a high-level “glue” language, Python is increasingly being used for rapid visualization of big datasets and to coordinate large-scale computations that run in compiled lan-

guages like C and FORTRAN. It's now relatively common to deal with datasets hundreds of gigabytes or even terabytes in size; HDF5 itself can scale up to exabytes.

On all but the biggest machines, it's not feasible to load such datasets directly into memory. One of HDF5's greatest strengths is its support for subsetting and partial I/O. For example, let's take the 1024-element "temperature" dataset we created earlier:

```
>>> dataset = f["/15/temperature"]
```

Here, the object named `dataset` is a proxy object representing an HDF5 dataset. It supports array-like slicing operations, which will be familiar to frequent NumPy users:

```
>>> dataset[0:10]
array([ 0.44149738,  0.7407523 ,  0.44243584,  0.3100173 ,  0.04552416,
        0.43933469,  0.28550775,  0.76152561,  0.79451732,  0.32603454])
>>> dataset[0:10:2]
array([ 0.44149738,  0.44243584,  0.04552416,  0.28550775,  0.79451732])
```

Keep in mind that the actual data lives on disk; when slicing is applied to an HDF5 dataset, the appropriate data is found and loaded into memory. Slicing in this fashion leverages the underlying subsetting capabilities of HDF5 and is consequently very fast.

Another great thing about HDF5 is that you have control over how storage is allocated. For example, except for some metadata, a brand new dataset takes *zero* space, and by default bytes are only used on disk to hold the data you actually write.

For example, here's a 2-terabyte dataset you can create on just about any computer:

```
>>> big_dataset = f.create_dataset("big", shape=(1024, 1024, 1024, 512),
dtype='float32')
```

Although no storage is yet allocated, the entire "space" of the dataset is available to us. We can write anywhere in the dataset, and only the bytes on disk necessary to hold the data are used:

```
>>> big_dataset[344, 678, 23, 36] = 42.0
```

When storage is at a premium, you can even use transparent compression on a dataset-by-dataset basis (see [Chapter 4](#)):

```
>>> compressed_dataset = f.create_dataset("comp", shape=(1024,), dtype='int32',
compression='gzip')
>>> compressed_dataset[:] = np.arange(1024)
>>> compressed_dataset[:]
array([ 0, 1, 2, ..., 1021, 1022, 1023])
```

What Exactly Is HDF5?

HDF5 is a great mechanism for storing *large numerical arrays of homogenous type*, for data models that can be *organized hierarchically* and benefit from tagging of datasets with *arbitrary metadata*.

It's quite different from SQL-style relational databases. HDF5 has quite a few organizational tricks up its sleeve (see [Chapter 8](#), for example), but if you find yourself needing to enforce relationships between values in various tables, or wanting to perform JOINS on your data, a relational database is probably more appropriate. Likewise, for tiny 1D datasets you need to be able to read on machines without HDF5 installed. Text formats like CSV (with all their warts) are a reasonable alternative.

HDF5 is just about perfect if you make minimal use of relational features and have a need for very high performance, partial I/O, hierarchical organization, and arbitrary metadata.

So what, specifically, is “HDF5”? I would argue it consists of three things:

1. A file specification and associated data model.
2. A standard library with API access available from C, C++, Java, Python, and others.
3. A software ecosystem, consisting of both client programs using HDF5 and “analysis platforms” like MATLAB, IDL, and Python.

HDF5: The File

In the preceding brief examples, you saw the three main elements of the HDF5 data model: *datasets*, array-like objects that store your numerical data on disk; *groups*, hierarchical containers that store datasets and other groups; and *attributes*, user-defined bits of metadata that can be attached to datasets (and groups!).

Using these basic abstractions, users can build specific “application formats” that organize data in a method appropriate for the problem domain. For example, our “weather station” code used one group for each station, and separate datasets for each measured parameter, with attributes to hold additional information about what the datasets mean. It's very common for laboratories or other organizations to agree on such a “format-within-a-format” that specifies what arrangement of groups, datasets, and attributes are to be used to store information.

Since HDF5 takes care of all cross-platform issues like endianness, sharing data with other groups becomes a simple matter of manipulating groups, datasets, and attributes to get the desired result. And because the files are *self-describing*, even knowing about the application format isn't usually necessary to get data out of the file. You can simply open the file and explore its contents:

```
>>> f.keys()
[u'15', u'big', u'comp']
>>> f["/15"].keys()
[u'temperature', u'wind']
```

Anyone who has spent hours fiddling with byte-offsets while trying to read “simple” binary formats can appreciate this.

Finally, the low-level byte layout of an HDF5 file on disk is an open specification. There are no mysteries about how it works, in contrast to proprietary binary formats. And although people typically use the library provided by the HDF Group to access files, nothing prevents you from writing your own reader if you want.

HDF5: The Library

The HDF5 file specification and open source library is maintained by the **HDF Group**, a nonprofit organization headquartered in Champaign, Illinois. Formerly part of the University of Illinois Urbana-Champaign, the HDF Group’s primary product is the HDF5 software library.

Written in C, with additional bindings for C++ and Java, this library is what people usually mean when they say “HDF5.” Both of the most popular Python interfaces, PyTables and h5py, are designed to use the C library provided by the HDF Group.

One important point to make is that this library is actively maintained, and the developers place a strong emphasis on backwards compatibility. This applies to both the files the library produces and also to programs that use the API. File compatibility is a must for an archival format like HDF5. Such careful attention to API compatibility is the main reason that packages like h5py and PyTables have been able to get traction with many different versions of HDF5 installed in the wild.

You should have confidence when using HDF5 for scientific data storage, including long-term storage. And since both the library and format are open source, your files will be readable even if a meteor takes out Illinois.

HDF5: The Ecosystem

Finally, one aspect that makes HDF5 particularly useful is that you can read and write files from just about every platform. The IDL language has supported HDF5 for years; MATLAB has similar support and now even uses HDF5 as the default format for its “.mat” save files. Bindings are also available for Python, C++, Java, .NET, and LabView, among others. Institutional users include NASA’s Earth Observing System, whose “EOS5” format is an application format on top of the HDF5 container, as in the much simpler example earlier. Even the newest version of the competing NetCDF format, NetCDF4, is implemented using HDF5 groups, datasets, and attributes.

Hopefully I’ve been able to share with you some of the things that make HDF5 so exciting for scientific use. Next, we’ll review the basics of how HDF5 works and get started on using it from Python.

HDF5 Basics

Before we jump into Python code examples, it's useful to take a few minutes to address how HDF5 itself is organized. **Figure 2-1** shows a cartoon of the various logical layers involved when using HDF5. Layers shaded in blue are internal to the library itself; layers in green represent software that uses HDF5.

Most client code, including the Python packages `h5py` and `PyTables`, uses the native C API (HDF5 is itself written in C). As we saw in the introduction, the HDF5 data model consists of three main *public abstractions*: datasets (see **Chapter 3**), groups (see **Chapter 5**), and attributes (see **Chapter 6**) in addition to a system to represent types. The C API (and Python code on top of it) is designed to manipulate these objects.

HDF5 uses a variety of *internal data structures* to represent groups, datasets, and attributes. For example, groups have their entries indexed using structures called “B-trees,” which make retrieving and creating group members very fast, even when hundreds of thousands of objects are stored in a group (see **“How Groups Are Actually Stored” on page 65**). You'll generally only care about these data structures when it comes to performance considerations. For example, when using chunked storage (see **Chapter 4**), it's important to understand how data is actually organized on disk.

The next two layers have to do with how your data makes its way onto disk. HDF5 objects all live in a 1D logical address space, like in a regular file. However, there's an extra layer between this space and the actual arrangement of bytes on disk. HDF5 *drivers* take care of the mechanics of writing to disk, and in the process can do some amazing things.

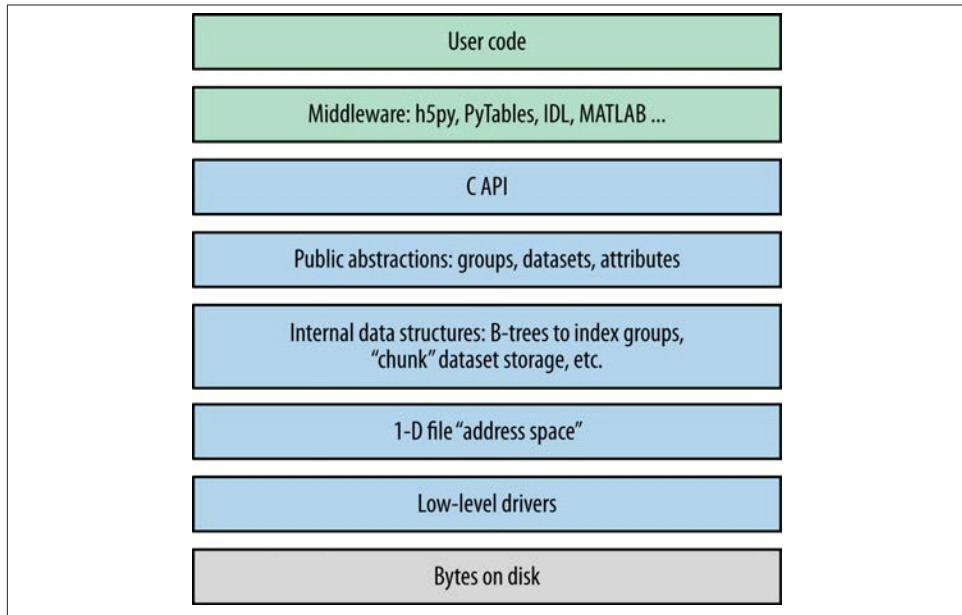


Figure 2-1. The HDF5 library: blue represents components inside the HDF5 library; green represents “client” code that calls into HDF5; gray represents resources provided by the operating system.

For example, the HDF5 core driver lets you use files that live entirely in memory and are blazingly fast. The family driver lets you split a single file into regularly sized pieces. And the mpio driver lets you access the same file from multiple parallel processes, using the Message Passing Interface (MPI) library (“[MPI and Parallel HDF5](#)” on page 119). All of this is transparent to code that works at the higher level of groups, datasets, and attributes.

Setting Up

That’s enough background material for now. Let’s get started with Python! But *which* Python?

Python 2 or Python 3?

A big shift is under way in the Python community. Over the years, Python has accumulated a number of features and misfeatures that have been deemed undesirable. Examples range from packages that use inconsistent naming conventions all the way to deficiencies in how strings are handled. To address these issues, it was decided to launch a new major version (Python 3) that would be freed from the “baggage” of old decisions in the Python 2 line.

Python 2.7, the most recent minor release in the Python series, will also be the *last* 2.X release. Although it will be updated with bug fixes for an extended period of time, new Python code development is now carried out exclusively on the 3.X line. The NumPy package, h5py, PyTables, and many other packages now support Python 3. While (in my opinion) it's a little early to recommend that newcomers start with Python 3, the future is clear.

So at the moment, there are two major versions of Python widely deployed simultaneously. Since most people in the Python community are used to Python 2, the examples in this book are also written for Python 2. For the most part, the differences are minor; for example, on Python 3, the syntax for `print` is `print(foo)`, instead of `print foo`. Wherever incompatibilities are likely to occur (mainly with string types and certain dictionary-style methods), these will be noted in the text.

“Porting” code to Python 3 isn't actually that hard; after all, it's still Python. Some of the most valuable features in Python 3 are already present in Python 2.7. A free tool is also available (2to3) that can accomplish most of the mechanical changes, for example changing `print` statements to `print()` function calls. Check out the migration guide (and the 2to3 tool) at <http://www.python.org>.

Code Examples

To start with, most of the code examples will follow this form:

```
>>> a = 1.0
>>> print a
1.0
```

Or, since Python prints objects by default, an even simpler form:

```
>>> a
1.0
```

Lines starting with `>>>` represent input to Python (`>>>` being the default Python prompt); other lines represent output. Some of the longer examples, where the program output is not shown, omit the `>>>` in the interest of clarity.

Examples intended to be run from the command line will start with the Unix-style “\$” prompt:

```
$ python --version
Python 2.7.3
```

Finally, to avoid cluttering up the examples, most of the code snippets you'll find here will assume that the following packages have been imported:

```
>>> import numpy as np          # Provides array object and data type objects
>>> import h5py                 # Provides access to HDF5
```

NumPy

“NumPy” is the standard numerical-array package in the Python world. This book assumes that you have some experience with NumPy (and of course Python itself), including how to use array objects.

Even if you’ve used NumPy before, it’s worth reviewing a few basic facts about how arrays work. First, NumPy arrays all have a fixed data type or “dtype,” represented by *dtype objects*. For example, let’s create a simple 10-element integer array:

```
>>> arr = np.arange(10)
>>> arr
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

The data type of this array is given by `arr.dtype`:

```
>>> arr.dtype
dtype('int32')
```

These dtype objects are how NumPy communicates low-level information about the type of data in memory. For the case of our 10-element array of 4-byte integers (32-bit or “int32” in NumPy lingo), there is a chunk of memory somewhere 40 bytes long that holds the values 0 through 9. Other code that receives the `arr` object can inspect the dtype object to figure out what the memory contents represent.



The preceding example might print `dtype('int64')` on your system. All this means is that the default integer size available to Python is 64 bits long, instead of 32 bits.

HDF5 uses a very similar type system; every “array” or *dataset* in an HDF5 file has a fixed type represented by a type object. The `h5py` package automatically maps the HDF5 type system onto NumPy dtypes, which among other things makes it easy to interchange data with NumPy. [Chapter 3](#) goes into more detail about this process.

Slicing is another core feature of NumPy. This means accessing *portions* of a NumPy array. For example, to extract the first four elements of our array `arr`:

```
>>> out = arr[0:4]
>>> out
array([0, 1, 2, 3])
```

You can also specify a “stride” or steps between points in the slice:

```
>>> out = arr[0:4:2]
>>> out
array([0, 2])
```

When talking to HDF5, we will borrow this “slicing” syntax to allow loading only portions of a dataset.

In the NumPy world, slicing is implemented in a clever way, which generally creates arrays that *refer* to the data in the original array, rather than independent copies. For example, the preceding `out` object is likely a “view” onto the original `arr` array. We can test this:

```
>>> out[:] = 42
>>> out
array([42, 42])
>>> arr
array([42, 1, 42, 3, 4, 5, 6, 7, 8, 9])
```

This means slicing is generally a very fast operation. But you should be careful to explicitly create a copy if you want to modify the resulting array without your changes finding their way back to the original:

```
>>> out2 = arr[0:4:2].copy()
```



Forgetting to make a copy before modifying a “slice” of the array is a very common mistake, especially for people used to environments like IDL. If you’re new to NumPy, be careful!

As we’ll see later, thankfully this doesn’t apply to slices read from HDF5 datasets. When you read from the file, since the data is on disk, you will always get a copy.

HDF5 and h5py

We’ll use the “h5py” package to talk to HDF5. This package contains high-level wrappers for the HDF5 objects of files, groups, datasets, and attributes, as well as extensive low-level wrappers for the HDF5 C data structures and functions. The examples in this book assume h5py version 2.2.0 or later, which you can get at <http://www.h5py.org>.

You should note that h5py is not the only commonly used package for talking to HDF5. **PyTables** is a scientific database package based on HDF5 that adds dataset indexing and an additional type system. Since we’ll be talking about native HDF5 constructs, we’ll stick to h5py, but I strongly recommend you also check out PyTables if those features interest you.

If you’re on Linux, it’s generally a good idea to install the HDF5 library via your package manager. On Windows, you can download an installer from <http://www.h5py.org>, or use one of the many distributions of Python that include HDF5/h5py, such as PythonXY, Anaconda from Continuum Analytics, or Enthought Python Distribution.

IPython

Apart from NumPy and h5py/HDF5 itself, IPython is a must-have component if you’ll be doing extensive analysis or development with Python. At its most basic, IPython is

a replacement interpreter shell that adds features like command history and Tab-completion for object attributes. It also has tons of additional features for parallel processing, MATLAB-style “notebook” display of graphs, and more.

The best way to explore the features in this book is with an IPython prompt open, following along with the examples. Tab-completion alone is worth it, because it lets you quickly see the attributes of modules and objects. The `h5py` package is specifically designed to be “explorable” in this sense. For example, if you want to discover what properties and methods exist on the `File` object (see “Your First HDF5 File” on page 17), type `h5py.File.` and bang the Tab key:

```
>>> h5py.File.<TAB>
h5py.File.attrs          h5py.File.get           h5py.File.name
h5py.File.close          h5py.File.id            h5py.File.parent
h5py.File.copy           h5py.File.items         h5py.File.ref
h5py.File.create_dataset h5py.File.iteritems     h5py.File.require_dataset
h5py.File.create_group   h5py.File.iterkeys      h5py.File.require_group
h5py.File.driver         h5py.File.itervalues    h5py.File.userblock_size
h5py.File.fid            h5py.File.keys          h5py.File.values
h5py.File.file           h5py.File.libver        h5py.File.visit
h5py.File.filename       h5py.File.mode          h5py.File.visititems
h5py.File.flush          h5py.File.mro
```

To get more information on a property or method, use `?` after its name:

```
>>> h5py.File.close?
Type:          instancemethod
Base Class:    <type 'instancemethod'>
String Form:   <unbound method File.close>
Namespace:    Interactive
File:         /usr/local/lib/python2.7/dist-packages/h5py/_hl/files.py
Definition:   h5py.File.close(self)
Docstring:    Close the file. All open objects become invalid
```



By default, IPython will save the output of your statements in special hidden variables. This is generally OK, but can be surprising if it hangs on to an HDF5 object you thought was discarded, or a big array that eats up memory. You can turn this off by setting the IPython configuration value `cache_size` to 0. See the docs at <http://ipython.org> for more information.

Timing and Optimization

For performance testing, we’ll use the `timeit` module that ships with Python. Examples using `timeit` will assume the following import:

```
>>> from timeit import timeit
```

The `timeit` function takes a (string or callable) command to execute, and an optional number of times it should be run. It then prints the total time spent running the command. For example, if we execute the “wait” function `time.sleep` five times:

```
>>> import time
>>> timeit("time.sleep(0.1)", number=5)
0.49967818316434887
```

If you’re using IPython, there’s a similar built-in “magic” function called `%timeit` that runs the specified statement a few times, and reports the lowest execution time:

```
>>> %timeit time.sleep(0.1)
10 loops, best of 3: 100 ms per loop
```

We’ll stick with the regular `timeit` function in this book, in part because it’s provided by the Python standard library.

Since people using HDF5 generally deal with large datasets, performance is always a concern. But you’ll notice that optimization and benchmarking discussions in this book don’t go into great detail about things like cache hits, data conversion rates, and so forth. The design of the `h5py` package, which this book uses, leaves nearly all of that to HDF5. This way, you benefit from the hundreds of man years of work spent on tuning HDF5 to provide the highest performance possible.

As an application builder, the best thing you can do for performance is to use the API in a sensible way and let HDF5 do its job. Here are some suggestions:

1. Don’t optimize anything unless there’s a demonstrated performance problem. Then, carefully isolate the misbehaving parts of the code before changing anything.
2. Start with simple, straightforward code that takes advantage of the API features. For example, to iterate over all objects in a file, use the Visitor feature of HDF5 (see [“Multilevel Iteration with the Visitor Pattern” on page 68](#)) rather than cobbling together your own approach.
3. Do “algorithmic” improvements first. For example, when writing to a dataset (see [Chapter 3](#)), write data in reasonably sized blocks instead of point by point. This lets HDF5 use the filesystem in an intelligent way.
4. Make sure you’re using the right data types. For example, if you’re running a compute-intensive program that loads floating-point data from a file, make sure that you’re not accidentally using double-precision floats in a calculation where single precision would do.
5. Finally, don’t hesitate to ask for help on the `h5py` or NumPy/Scipy mailing lists, Stack Overflow, or other community sites. Lots of people are using NumPy and HDF5 these days, and many performance problems have known solutions. The Python community is very welcoming.

The HDF5 Tools

We'll be creating a number of files in later chapters, and it's nice to have an independent way of seeing what they contain. It's also a good idea to inspect files you create professionally, especially if you'll be using them for archiving or sharing them with colleagues. The earlier you can detect the use of an incorrect data type, for example, the better off you and other users will be.

HDFView

HDFView is a free graphical browser for HDF5 files provided by the HDF Group. It's a little basic, but is written in Java and therefore available on Windows, Linux, and Mac. There's a built-in spreadsheet-like browser for data, and basic plotting capabilities.

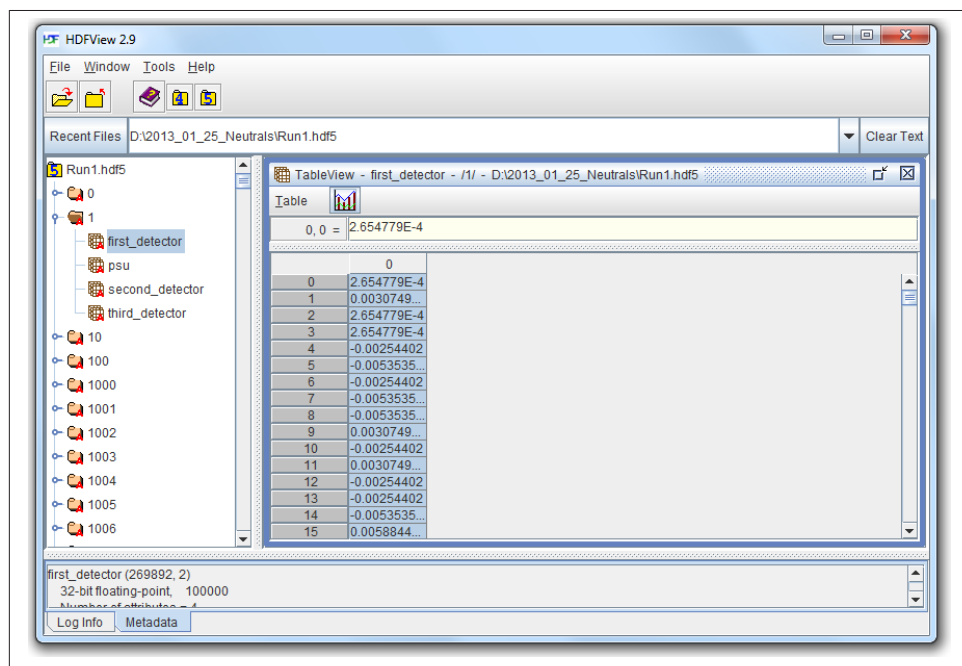


Figure 2-2. HDFView

Figure 2-2 shows the contents of an HDF5 file with multiple groups in the left-hand pane. One group (named “1”) is open, showing the datasets it contains; likewise, one dataset is opened, with its contents displayed in the grid view to the right.

HDFView also lets you inspect attributes of datasets and groups, and supports nearly all the data types that HDF5 itself supports, with the exception of certain variable-length structures.

ViTables

Figure 2-3 shows the same HDF5 file open in ViTables, another free graphical browser. It's optimized for dealing with PyTables files, although it can handle generic HDF5 files perfectly well. One major advantage of ViTables is that it comes preinstalled with such Python distributions as PythonXY, so you may already have it.

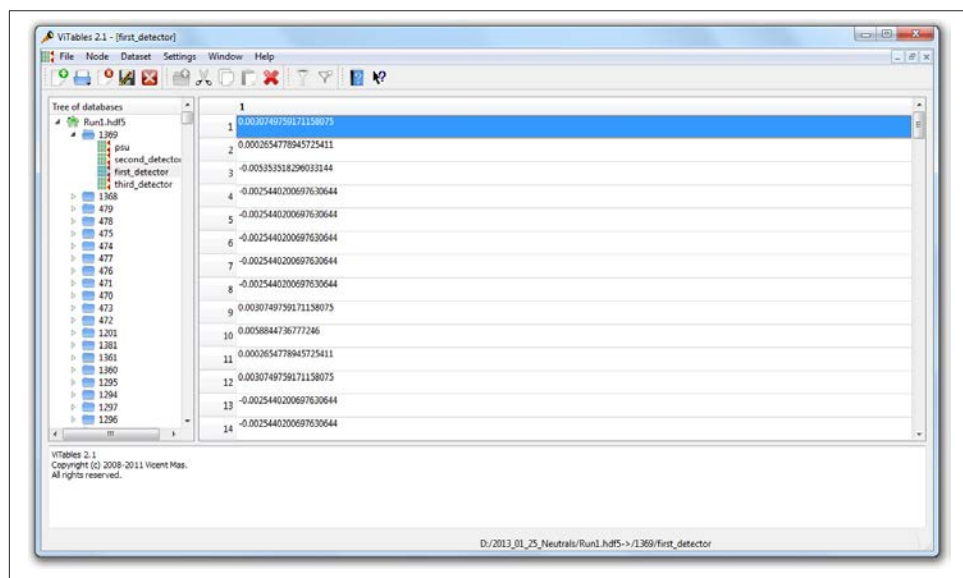


Figure 2-3. ViTables

Command Line Tools

If you're used to the command line, it's definitely worth installing the HDF command-line tools. These are generally available through a package manager; if not, you can get them at www.hdfgroup.org. Windows versions are also available.

The program we'll be using most in this book is called `h5ls`, which as the name suggests lists the contents of a file. Here's an example, in which `h5ls` is applied to a file containing a couple of datasets and a group:

```
$ h5ls demo.hdf5
array                Dataset {10}
group                Group
scalar               Dataset {SCALAR}
```

We can get a little more useful information by using the option combo `-vlr`, which prints extended information and also recursively enters groups:


```

$ h5ls -vlr demo.hdf5
/
  Location: 1:96
  Links: 1
/array
  Location: 1:1400
  Links: 1
  Storage: 40 logical bytes, 40 allocated bytes, 100.00% utilization
  Type: native int
/group
  Location: 1:1672
  Links: 1
/group/subarray
  Location: 1:1832
  Links: 1
  Storage: 16 logical bytes, 16 allocated bytes, 100.00% utilization
  Type: native int
/scalar
  Location: 1:800
  Links: 1
  Storage: 4 logical bytes, 4 allocated bytes, 100.00% utilization
  Type: native int

```

That’s a little more useful. We can see that the object at `/array` is of type “native int,” and is a 1D array 10 elements long. Likewise, there’s a dataset inside the group named `group` that is 2D, also of type native int.

`h5ls` is great for inspecting metadata like this. There’s also a program called `h5dump`, which prints data as well, although in a more verbose format:

```

$ h5dump demo.hdf5
HDF5 "demo.hdf5" {
GROUP "/" {
  DATASET "array" {
    DATATYPE  H5T_STD_I32LE
    DATASPACE  SIMPLE { ( 10 ) / ( 10 ) }
    DATA {
      (0): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
    }
  }
  GROUP "group" {
    DATASET "subarray" {
      DATATYPE  H5T_STD_I32LE
      DATASPACE  SIMPLE { ( 2, 2 ) / ( 2, 2 ) }
      DATA {
        (0,0): 2, 2,
        (1,0): 2, 2
      }
    }
  }
}
  DATASET "scalar" {
    DATATYPE  H5T_STD_I32LE

```

```

        DATASPACE SCALAR
        DATA {
            (0): 42
        }
    }
}
}

```

Your First HDF5 File

Before we get to groups and datasets, let's start by exploring some of the capabilities of the File object, which will be your entry point into the world of HDF5.

Here's the simplest possible program that uses HDF5:

```

>>> f = h5py.File("name.hdf5")
>>> f.close()

```

The File object is your starting point; it has methods that let you create new datasets or groups in the file, as well as more pedestrian properties such as `.filename` and `.mode`.

Speaking of `.mode`, HDF5 files support the same kind of read/write modes as regular Python files:

```

>>> f = h5py.File("name.hdf5", "w") # New file overwriting any existing file
>>> f = h5py.File("name.hdf5", "r") # Open read-only (must exist)
>>> f = h5py.File("name.hdf5", "r+") # Open read-write (must exist)
>>> f = h5py.File("name.hdf5", "a") # Open read-write (create if doesn't exist)

```

There's one additional HDF5-specific mode, which can save your bacon should you accidentally try to overwrite an existing file:

```

>>> f = h5py.File("name.hdf5", "w-")

```

This will create a new file, but fail if a file of the same name already exists. For example, if you're performing a long-running calculation and don't want to risk overwriting your output file should the script be run twice, you could open the file in `w-` mode:

```

>>> f = h5py.File("important_file.hdf5", "w-")
>>> f.close()
>>> f = h5py.File("important_file.hdf5", "w-")
IOError: unable to create file (File accessibility: Unable to open file)

```

By the way, you're free to use Unicode filenames! Just supply a normal Unicode string and it will transparently work, assuming the underlying operating system supports the UTF-8 encoding:

```

>>> name = u"name_eta_\u03b7"
>>> f = h5py.File(name)
>>> print f.filename
name_eta_η

```



You might wonder what happens if your program crashes with open files. If the program exits with a Python exception, don't worry! The HDF library will automatically close every open file for you when the application exits.

Use as a Context Manager

One of the coolest features introduced in Python 2.6 is support for *context managers*. These are objects with a few special methods called on entry and exit from a block of code, using the `with` statement. The classic example is the built-in Python file object:

```
>>> with open("somefile.txt", "w") as f:
...     f.write("Hello!")
```

The preceding code opens a brand-new file object, which is available in the code block with the name `f`. When the block exits, the file is automatically closed (even if an exception occurs!).

The `h5py.File` object supports exactly this kind of use. It's a great way to make sure the file is always properly closed, without wrapping everything in `try/except` blocks:

```
>>> with h5py.File("name.hdf5", "w") as f:
...     print f["missing_dataset"]
KeyError: "unable to open object (Symbol table: Can't open object)"
>>> print f
<Closed HDF5 file>
```

File Drivers

File drivers sit between the filesystem and the higher-level world of HDF5 groups, datasets, and attributes. They deal with the mechanics of mapping the HDF5 “address space” to an arrangement of bytes on disk. Typically you won't have to worry about which driver is in use, as the default driver works well for most applications.

The great thing about drivers is that once the file is opened, they're totally transparent. You just use the HDF5 library as normal, and the driver takes care of the storage mechanics.

Here are a couple of the more interesting ones, which can be helpful for unusual problems.

core driver

The core driver stores your file *entirely in memory*. Obviously there's a limit to how much data you can store, but the trade-off is blazingly fast reads and writes. It's a great choice when you want the speed of memory access, but also want to use the HDF5 structures. To enable, set the driver keyword to "core":

```
>>> f = h5py.File("name.hdf5", driver="core")
```

You can also tell HDF5 to create an on-disk “backing store” file, to which the file image is saved when closed:

```
>>> f = h5py.File("name.hdf5", driver="core", backing_store=True)
```

By the way, the `backing_store` keyword will also tell HDF5 to load any existing image from disk when you open the file. So if the entire file will fit in memory, you need to read and write the image only once; things like dataset reads and writes, attribute creation, and so on, don’t take any disk I/O at all.

family driver

Sometimes it’s convenient to split a file up into multiple images, all of which share a certain maximum size. This feature was originally implemented to support filesystems that couldn’t handle file sizes above 2GB.

```
>>> # Split the file into 1-GB chunks
>>> f = h5py.File("family.hdf5", driver="family", memb_size=1024**3)
```

The default for `memb_size` is $2^{31}-1$, in keeping with the historical origins of the driver.

mpio driver

This driver is the heart of Parallel HDF5. It lets you access the same file from multiple processes at the same time. You can have dozens or even hundreds of parallel computing processes, all of which share a consistent view of a single file on disk.

Using the `mpio` driver correctly can be tricky. [Chapter 9](#) covers both the details of this driver and best practices for using HDF5 in a parallel environment.

The User Block

One interesting feature of HDF5 is that files may be preceded by arbitrary user data. When a file is opened, the library looks for the HDF5 header at the beginning of the file, then 512 bytes in, then 1024, and so on in powers of 2. Such space at the beginning of the file is called the “user block,” and you can store whatever data you want there.

The only restrictions are on the size of the block (powers of 2, and at least 512), and that you shouldn’t have the file open in HDF5 when writing to the user block. Here’s an example:

```
>>> f = h5py.File("userblock.hdf5", "w", userblock_size=512)
>>> f.userblock_size    # Would be 0 if no user block present
512
>>> f.close()

>>> with open("userblock.hdf5", "rb+") as f:    # Open as regular Python file
...     f.write("a"*512)
```

Let's move on to the first major object in the HDF5 data model, one that will be familiar to users of the NumPy array type: datasets.

HDF5 Basics

Before we jump into Python code examples, it's useful to take a few minutes to address how HDF5 itself is organized. **Figure 2-1** shows a cartoon of the various logical layers involved when using HDF5. Layers shaded in blue are internal to the library itself; layers in green represent software that uses HDF5.

Most client code, including the Python packages `h5py` and `PyTables`, uses the native C API (HDF5 is itself written in C). As we saw in the introduction, the HDF5 data model consists of three main *public abstractions*: datasets (see **Chapter 3**), groups (see **Chapter 5**), and attributes (see **Chapter 6**) in addition to a system to represent types. The C API (and Python code on top of it) is designed to manipulate these objects.

HDF5 uses a variety of *internal data structures* to represent groups, datasets, and attributes. For example, groups have their entries indexed using structures called “B-trees,” which make retrieving and creating group members very fast, even when hundreds of thousands of objects are stored in a group (see **“How Groups Are Actually Stored” on page 65**). You'll generally only care about these data structures when it comes to performance considerations. For example, when using chunked storage (see **Chapter 4**), it's important to understand how data is actually organized on disk.

The next two layers have to do with how your data makes its way onto disk. HDF5 objects all live in a 1D logical address space, like in a regular file. However, there's an extra layer between this space and the actual arrangement of bytes on disk. HDF5 *drivers* take care of the mechanics of writing to disk, and in the process can do some amazing things.

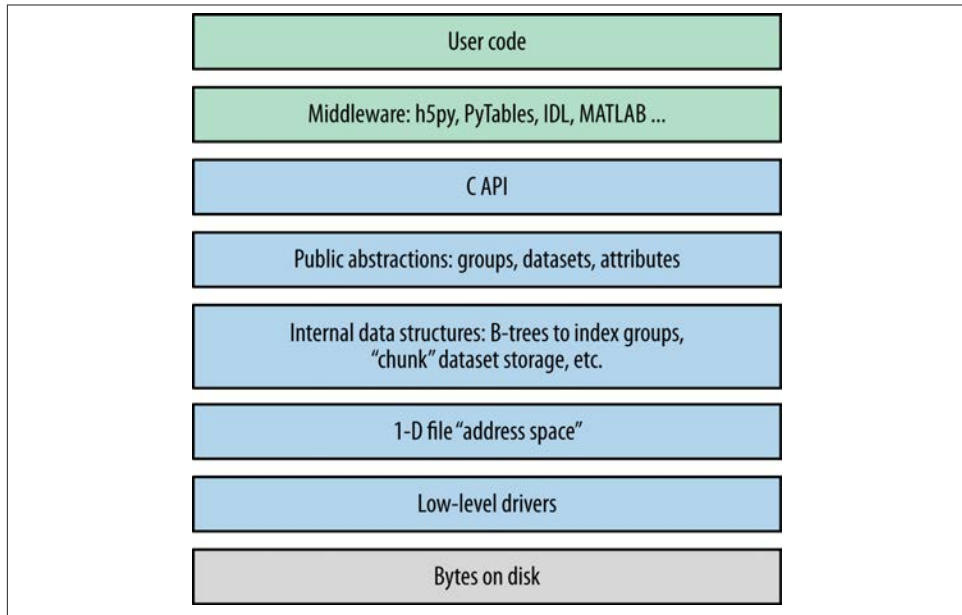


Figure 2-1. The HDF5 library: blue represents components inside the HDF5 library; green represents “client” code that calls into HDF5; gray represents resources provided by the operating system.

For example, the HDF5 core driver lets you use files that live entirely in memory and are blazingly fast. The family driver lets you split a single file into regularly sized pieces. And the mpio driver lets you access the same file from multiple parallel processes, using the Message Passing Interface (MPI) library ([“MPI and Parallel HDF5” on page 119](#)). All of this is transparent to code that works at the higher level of groups, datasets, and attributes.

Setting Up

That’s enough background material for now. Let’s get started with Python! But *which* Python?

Python 2 or Python 3?

A big shift is under way in the Python community. Over the years, Python has accumulated a number of features and misfeatures that have been deemed undesirable. Examples range from packages that use inconsistent naming conventions all the way to deficiencies in how strings are handled. To address these issues, it was decided to launch a new major version (Python 3) that would be freed from the “baggage” of old decisions in the Python 2 line.

Python 2.7, the most recent minor release in the Python series, will also be the *last* 2.X release. Although it will be updated with bug fixes for an extended period of time, new Python code development is now carried out exclusively on the 3.X line. The NumPy package, h5py, PyTables, and many other packages now support Python 3. While (in my opinion) it's a little early to recommend that newcomers start with Python 3, the future is clear.

So at the moment, there are two major versions of Python widely deployed simultaneously. Since most people in the Python community are used to Python 2, the examples in this book are also written for Python 2. For the most part, the differences are minor; for example, on Python 3, the syntax for `print` is `print(foo)`, instead of `print foo`. Wherever incompatibilities are likely to occur (mainly with string types and certain dictionary-style methods), these will be noted in the text.

“Porting” code to Python 3 isn't actually that hard; after all, it's still Python. Some of the most valuable features in Python 3 are already present in Python 2.7. A free tool is also available (2to3) that can accomplish most of the mechanical changes, for example changing `print` statements to `print()` function calls. Check out the migration guide (and the 2to3 tool) at <http://www.python.org>.

Code Examples

To start with, most of the code examples will follow this form:

```
>>> a = 1.0
>>> print a
1.0
```

Or, since Python prints objects by default, an even simpler form:

```
>>> a
1.0
```

Lines starting with `>>>` represent input to Python (`>>>` being the default Python prompt); other lines represent output. Some of the longer examples, where the program output is not shown, omit the `>>>` in the interest of clarity.

Examples intended to be run from the command line will start with the Unix-style “\$” prompt:

```
$ python --version
Python 2.7.3
```

Finally, to avoid cluttering up the examples, most of the code snippets you'll find here will assume that the following packages have been imported:

```
>>> import numpy as np           # Provides array object and data type objects
>>> import h5py                  # Provides access to HDF5
```


NumPy

“NumPy” is the standard numerical-array package in the Python world. This book assumes that you have some experience with NumPy (and of course Python itself), including how to use array objects.

Even if you’ve used NumPy before, it’s worth reviewing a few basic facts about how arrays work. First, NumPy arrays all have a fixed data type or “dtype,” represented by *dtype objects*. For example, let’s create a simple 10-element integer array:

```
>>> arr = np.arange(10)
>>> arr
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

The data type of this array is given by `arr.dtype`:

```
>>> arr.dtype
dtype('int32')
```

These dtype objects are how NumPy communicates low-level information about the type of data in memory. For the case of our 10-element array of 4-byte integers (32-bit or “int32” in NumPy lingo), there is a chunk of memory somewhere 40 bytes long that holds the values 0 through 9. Other code that receives the `arr` object can inspect the dtype object to figure out what the memory contents represent.



The preceding example might print `dtype('int64')` on your system. All this means is that the default integer size available to Python is 64 bits long, instead of 32 bits.

HDF5 uses a very similar type system; every “array” or *dataset* in an HDF5 file has a fixed type represented by a type object. The `h5py` package automatically maps the HDF5 type system onto NumPy dtypes, which among other things makes it easy to interchange data with NumPy. [Chapter 3](#) goes into more detail about this process.

Slicing is another core feature of NumPy. This means accessing *portions* of a NumPy array. For example, to extract the first four elements of our array `arr`:

```
>>> out = arr[0:4]
>>> out
array([0, 1, 2, 3])
```

You can also specify a “stride” or steps between points in the slice:

```
>>> out = arr[0:4:2]
>>> out
array([0, 2])
```

When talking to HDF5, we will borrow this “slicing” syntax to allow loading only portions of a dataset.

In the NumPy world, slicing is implemented in a clever way, which generally creates arrays that *refer* to the data in the original array, rather than independent copies. For example, the preceding `out` object is likely a “view” onto the original `arr` array. We can test this:

```
>>> out[:] = 42
>>> out
array([42, 42])
>>> arr
array([42, 1, 42, 3, 4, 5, 6, 7, 8, 9])
```

This means slicing is generally a very fast operation. But you should be careful to explicitly create a copy if you want to modify the resulting array without your changes finding their way back to the original:

```
>>> out2 = arr[0:4:2].copy()
```



Forgetting to make a copy before modifying a “slice” of the array is a very common mistake, especially for people used to environments like IDL. If you’re new to NumPy, be careful!

As we’ll see later, thankfully this doesn’t apply to slices read from HDF5 datasets. When you read from the file, since the data is on disk, you will always get a copy.

HDF5 and h5py

We’ll use the “h5py” package to talk to HDF5. This package contains high-level wrappers for the HDF5 objects of files, groups, datasets, and attributes, as well as extensive low-level wrappers for the HDF5 C data structures and functions. The examples in this book assume h5py version 2.2.0 or later, which you can get at <http://www.h5py.org>.

You should note that h5py is not the only commonly used package for talking to HDF5. **PyTables** is a scientific database package based on HDF5 that adds dataset indexing and an additional type system. Since we’ll be talking about native HDF5 constructs, we’ll stick to h5py, but I strongly recommend you also check out PyTables if those features interest you.

If you’re on Linux, it’s generally a good idea to install the HDF5 library via your package manager. On Windows, you can download an installer from <http://www.h5py.org>, or use one of the many distributions of Python that include HDF5/h5py, such as PythonXY, Anaconda from Continuum Analytics, or Enthought Python Distribution.

IPython

Apart from NumPy and h5py/HDF5 itself, IPython is a must-have component if you’ll be doing extensive analysis or development with Python. At its most basic, IPython is

a replacement interpreter shell that adds features like command history and Tab-completion for object attributes. It also has tons of additional features for parallel processing, MATLAB-style “notebook” display of graphs, and more.

The best way to explore the features in this book is with an IPython prompt open, following along with the examples. Tab-completion alone is worth it, because it lets you quickly see the attributes of modules and objects. The h5py package is specifically designed to be “explorable” in this sense. For example, if you want to discover what properties and methods exist on the File object (see “Your First HDF5 File” on page 17), type `h5py.File.` and bang the Tab key:

```
>>> h5py.File.<TAB>
h5py.File.attrs          h5py.File.get           h5py.File.name
h5py.File.close          h5py.File.id            h5py.File.parent
h5py.File.copy           h5py.File.items         h5py.File.ref
h5py.File.create_dataset h5py.File.iteritems     h5py.File.require_dataset
h5py.File.create_group   h5py.File.iterkeys      h5py.File.require_group
h5py.File.driver         h5py.File.itervalues    h5py.File.userblock_size
h5py.File.fid            h5py.File.keys          h5py.File.values
h5py.File.file           h5py.File.libver        h5py.File.visit
h5py.File.filename       h5py.File.mode          h5py.File.visititems
h5py.File.flush          h5py.File.mro
```

To get more information on a property or method, use `?` after its name:

```
>>> h5py.File.close?
Type:          instancemethod
Base Class:    <type 'instancemethod'>
String Form:   <unbound method File.close>
Namespace:     Interactive
File:          /usr/local/lib/python2.7/dist-packages/h5py/_hl/files.py
Definition:    h5py.File.close(self)
Docstring:     Close the file. All open objects become invalid
```



By default, IPython will save the output of your statements in special hidden variables. This is generally OK, but can be surprising if it hangs on to an HDF5 object you thought was discarded, or a big array that eats up memory. You can turn this off by setting the IPython configuration value `cache_size` to 0. See the docs at <http://ipython.org> for more information.

Timing and Optimization

For performance testing, we’ll use the `timeit` module that ships with Python. Examples using `timeit` will assume the following import:

```
>>> from timeit import timeit
```

The `timeit` function takes a (string or callable) command to execute, and an optional number of times it should be run. It then prints the total time spent running the command. For example, if we execute the “wait” function `time.sleep` five times:

```
>>> import time
>>> timeit("time.sleep(0.1)", number=5)
0.49967818316434887
```

If you’re using IPython, there’s a similar built-in “magic” function called `%timeit` that runs the specified statement a few times, and reports the lowest execution time:

```
>>> %timeit time.sleep(0.1)
10 loops, best of 3: 100 ms per loop
```

We’ll stick with the regular `timeit` function in this book, in part because it’s provided by the Python standard library.

Since people using HDF5 generally deal with large datasets, performance is always a concern. But you’ll notice that optimization and benchmarking discussions in this book don’t go into great detail about things like cache hits, data conversion rates, and so forth. The design of the `h5py` package, which this book uses, leaves nearly all of that to HDF5. This way, you benefit from the hundreds of man years of work spent on tuning HDF5 to provide the highest performance possible.

As an application builder, the best thing you can do for performance is to use the API in a sensible way and let HDF5 do its job. Here are some suggestions:

1. Don’t optimize anything unless there’s a demonstrated performance problem. Then, carefully isolate the misbehaving parts of the code before changing anything.
2. Start with simple, straightforward code that takes advantage of the API features. For example, to iterate over all objects in a file, use the Visitor feature of HDF5 (see [“Multilevel Iteration with the Visitor Pattern” on page 68](#)) rather than cobbling together your own approach.
3. Do “algorithmic” improvements first. For example, when writing to a dataset (see [Chapter 3](#)), write data in reasonably sized blocks instead of point by point. This lets HDF5 use the filesystem in an intelligent way.
4. Make sure you’re using the right data types. For example, if you’re running a compute-intensive program that loads floating-point data from a file, make sure that you’re not accidentally using double-precision floats in a calculation where single precision would do.
5. Finally, don’t hesitate to ask for help on the `h5py` or NumPy/Scipy mailing lists, Stack Overflow, or other community sites. Lots of people are using NumPy and HDF5 these days, and many performance problems have known solutions. The Python community is very welcoming.

The HDF5 Tools

We'll be creating a number of files in later chapters, and it's nice to have an independent way of seeing what they contain. It's also a good idea to inspect files you create professionally, especially if you'll be using them for archiving or sharing them with colleagues. The earlier you can detect the use of an incorrect data type, for example, the better off you and other users will be.

HDFView

HDFView is a free graphical browser for HDF5 files provided by the HDF Group. It's a little basic, but is written in Java and therefore available on Windows, Linux, and Mac. There's a built-in spreadsheet-like browser for data, and basic plotting capabilities.

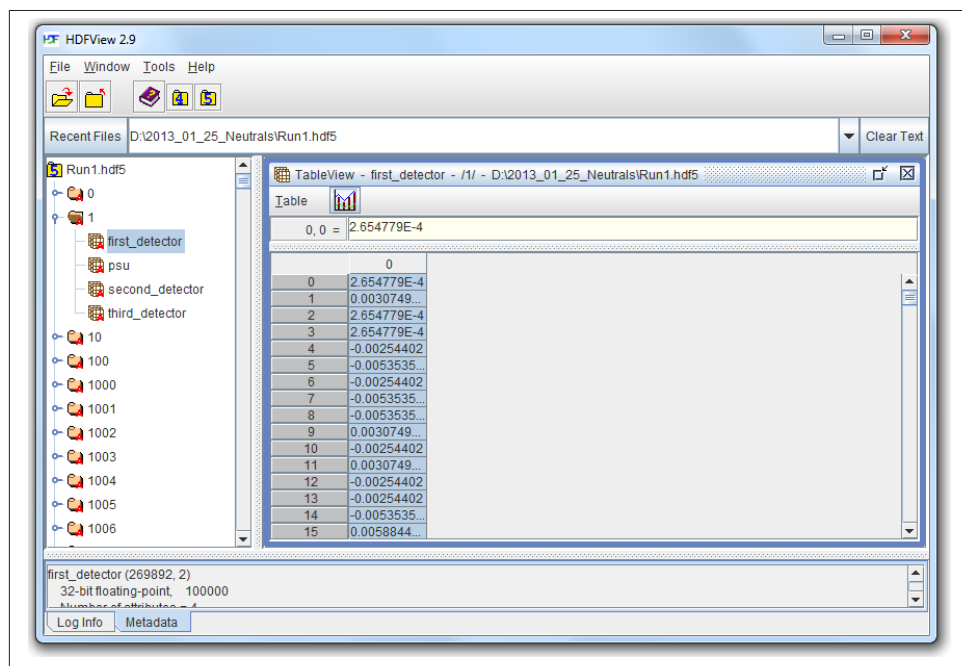


Figure 2-2. HDFView

Figure 2-2 shows the contents of an HDF5 file with multiple groups in the left-hand pane. One group (named “1”) is open, showing the datasets it contains; likewise, one dataset is opened, with its contents displayed in the grid view to the right.

HDFView also lets you inspect attributes of datasets and groups, and supports nearly all the data types that HDF5 itself supports, with the exception of certain variable-length structures.

ViTables

Figure 2-3 shows the same HDF5 file open in ViTables, another free graphical browser. It's optimized for dealing with PyTables files, although it can handle generic HDF5 files perfectly well. One major advantage of ViTables is that it comes preinstalled with such Python distributions as PythonXY, so you may already have it.

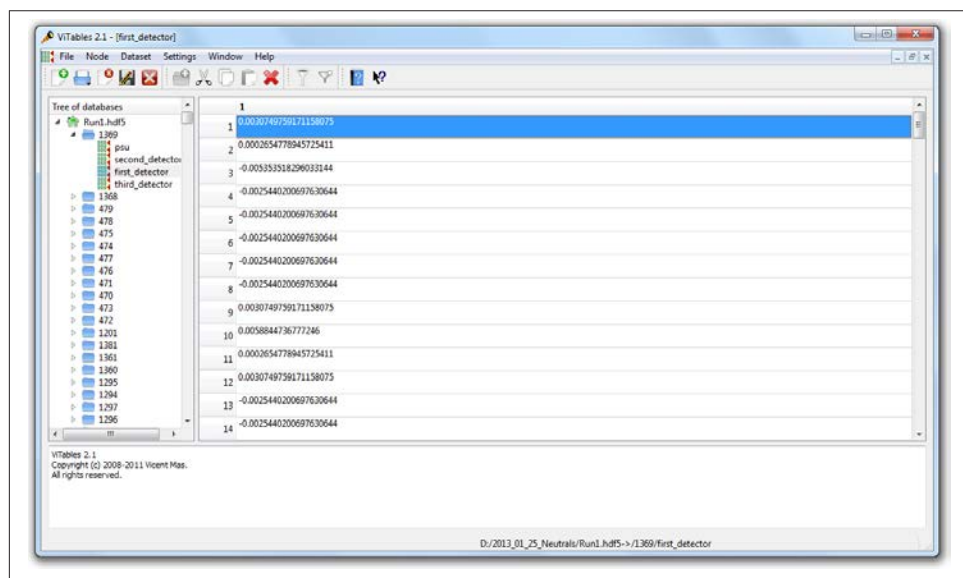


Figure 2-3. ViTables

Command Line Tools

If you're used to the command line, it's definitely worth installing the HDF command-line tools. These are generally available through a package manager; if not, you can get them at www.hdfgroup.org. Windows versions are also available.

The program we'll be using most in this book is called `h5ls`, which as the name suggests lists the contents of a file. Here's an example, in which `h5ls` is applied to a file containing a couple of datasets and a group:

```
$ h5ls demo.hdf5
array                Dataset {10}
group                Group
scalar               Dataset {SCALAR}
```

We can get a little more useful information by using the option combo `-vlr`, which prints extended information and also recursively enters groups:

```

$ h5ls -vlr demo.hdf5
/
  Location: 1:96
  Links: 1
/array
  Location: 1:1400
  Links: 1
  Storage: 40 logical bytes, 40 allocated bytes, 100.00% utilization
  Type: native int
/group
  Location: 1:1672
  Links: 1
/group/subarray
  Location: 1:1832
  Links: 1
  Storage: 16 logical bytes, 16 allocated bytes, 100.00% utilization
  Type: native int
/scalar
  Location: 1:800
  Links: 1
  Storage: 4 logical bytes, 4 allocated bytes, 100.00% utilization
  Type: native int

```

That’s a little more useful. We can see that the object at `/array` is of type “native int,” and is a 1D array 10 elements long. Likewise, there’s a dataset inside the group named `group` that is 2D, also of type native int.

`h5ls` is great for inspecting metadata like this. There’s also a program called `h5dump`, which prints data as well, although in a more verbose format:

```

$ h5dump demo.hdf5
HDF5 "demo.hdf5" {
  GROUP "/" {
    DATASET "array" {
      DATATYPE  H5T_STD_I32LE
      DATASPACE  SIMPLE { ( 10 ) / ( 10 ) }
      DATA {
        (0): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
      }
    }
    GROUP "group" {
      DATASET "subarray" {
        DATATYPE  H5T_STD_I32LE
        DATASPACE  SIMPLE { ( 2, 2 ) / ( 2, 2 ) }
        DATA {
          (0,0): 2, 2,
          (1,0): 2, 2
        }
      }
    }
  }
  DATASET "scalar" {
    DATATYPE  H5T_STD_I32LE

```

```

        DATASPACE SCALAR
        DATA {
            (0): 42
        }
    }
}
}

```

Your First HDF5 File

Before we get to groups and datasets, let's start by exploring some of the capabilities of the File object, which will be your entry point into the world of HDF5.

Here's the simplest possible program that uses HDF5:

```

>>> f = h5py.File("name.hdf5")
>>> f.close()

```

The File object is your starting point; it has methods that let you create new datasets or groups in the file, as well as more pedestrian properties such as `.filename` and `.mode`.

Speaking of `.mode`, HDF5 files support the same kind of read/write modes as regular Python files:

```

>>> f = h5py.File("name.hdf5", "w") # New file overwriting any existing file
>>> f = h5py.File("name.hdf5", "r") # Open read-only (must exist)
>>> f = h5py.File("name.hdf5", "r+") # Open read-write (must exist)
>>> f = h5py.File("name.hdf5", "a") # Open read-write (create if doesn't exist)

```

There's one additional HDF5-specific mode, which can save your bacon should you accidentally try to overwrite an existing file:

```

>>> f = h5py.File("name.hdf5", "w-")

```

This will create a new file, but fail if a file of the same name already exists. For example, if you're performing a long-running calculation and don't want to risk overwriting your output file should the script be run twice, you could open the file in `w-` mode:

```

>>> f = h5py.File("important_file.hdf5", "w-")
>>> f.close()
>>> f = h5py.File("important_file.hdf5", "w-")
IOError: unable to create file (File accessibility: Unable to open file)

```

By the way, you're free to use Unicode filenames! Just supply a normal Unicode string and it will transparently work, assuming the underlying operating system supports the UTF-8 encoding:

```

>>> name = u"name_eta_\u03b7"
>>> f = h5py.File(name)
>>> print f.filename
name_eta_η

```




You might wonder what happens if your program crashes with open files. If the program exits with a Python exception, don't worry! The HDF library will automatically close every open file for you when the application exits.

Use as a Context Manager

One of the coolest features introduced in Python 2.6 is support for *context managers*. These are objects with a few special methods called on entry and exit from a block of code, using the `with` statement. The classic example is the built-in Python file object:

```
>>> with open("somefile.txt", "w") as f:
...     f.write("Hello!")
```

The preceding code opens a brand-new file object, which is available in the code block with the name `f`. When the block exits, the file is automatically closed (even if an exception occurs!).

The `h5py.File` object supports exactly this kind of use. It's a great way to make sure the file is always properly closed, without wrapping everything in `try/except` blocks:

```
>>> with h5py.File("name.hdf5", "w") as f:
...     print f["missing_dataset"]
KeyError: "unable to open object (Symbol table: Can't open object)"
>>> print f
<Closed HDF5 file>
```

File Drivers

File drivers sit between the filesystem and the higher-level world of HDF5 groups, datasets, and attributes. They deal with the mechanics of mapping the HDF5 “address space” to an arrangement of bytes on disk. Typically you won't have to worry about which driver is in use, as the default driver works well for most applications.

The great thing about drivers is that once the file is opened, they're totally transparent. You just use the HDF5 library as normal, and the driver takes care of the storage mechanics.

Here are a couple of the more interesting ones, which can be helpful for unusual problems.

core driver

The core driver stores your file *entirely in memory*. Obviously there's a limit to how much data you can store, but the trade-off is blazingly fast reads and writes. It's a great choice when you want the speed of memory access, but also want to use the HDF5 structures. To enable, set the driver keyword to "core":

```
>>> f = h5py.File("name.hdf5", driver="core")
```

You can also tell HDF5 to create an on-disk “backing store” file, to which the file image is saved when closed:

```
>>> f = h5py.File("name.hdf5", driver="core", backing_store=True)
```

By the way, the `backing_store` keyword will also tell HDF5 to load any existing image from disk when you open the file. So if the entire file will fit in memory, you need to read and write the image only once; things like dataset reads and writes, attribute creation, and so on, don’t take any disk I/O at all.

family driver

Sometimes it’s convenient to split a file up into multiple images, all of which share a certain maximum size. This feature was originally implemented to support filesystems that couldn’t handle file sizes above 2GB.

```
>>> # Split the file into 1-GB chunks
>>> f = h5py.File("family.hdf5", driver="family", memb_size=1024**3)
```

The default for `memb_size` is $2^{31}-1$, in keeping with the historical origins of the driver.

mpio driver

This driver is the heart of Parallel HDF5. It lets you access the same file from multiple processes at the same time. You can have dozens or even hundreds of parallel computing processes, all of which share a consistent view of a single file on disk.

Using the `mpio` driver correctly can be tricky. [Chapter 9](#) covers both the details of this driver and best practices for using HDF5 in a parallel environment.

The User Block

One interesting feature of HDF5 is that files may be preceded by arbitrary user data. When a file is opened, the library looks for the HDF5 header at the beginning of the file, then 512 bytes in, then 1024, and so on in powers of 2. Such space at the beginning of the file is called the “user block,” and you can store whatever data you want there.

The only restrictions are on the size of the block (powers of 2, and at least 512), and that you shouldn’t have the file open in HDF5 when writing to the user block. Here’s an example:

```
>>> f = h5py.File("userblock.hdf5", "w", userblock_size=512)
>>> f.userblock_size    # Would be 0 if no user block present
512
>>> f.close()

>>> with open("userblock.hdf5", "rb+") as f:    # Open as regular Python file
...     f.write("a"*512)
```

Let's move on to the first major object in the HDF5 data model, one that will be familiar to users of the NumPy array type: datasets.

Groups, Links, and Iteration: The “H” in HDF5

So far we’ve seen how to create `Dataset` objects by giving them a name in the file like `myfile["dataset1"]` or `myfile["dataset2"]`. Unless you’re one of those people who stores all their documents on the desktop, you can probably see the flaw in this approach.

Groups are the HDF5 container object, analagous to folders in a filesystem. They can hold datasets and other groups, allowing you to build up a hierarchical structure with objects neatly organized in groups and subgroups.

The Root Group and Subgroups

You may have guessed by now that the `File` object is itself a group. In this case, it also serves as the *root group*, named `/`, our entry point into the file.

The more general group object is `h5py.Group`, of which `h5py.File` is a subclass. Other groups are easily created by the method `create_group`:

```
>>> f = h5py.File("Groups.hdf5")
>>> subgroup = f.create_group("SubGroup")
>>> subgroup
<HDF5 group "/SubGroup" (0 members)>
>>> subgroup.name
u'/SubGroup'
```

Of course, groups can be nested also. The `create_group` method exists on all `Group` objects, not just `File`:

```
>>> subsubgroup = subgroup.create_group("AnotherGroup")
>>> subsubgroup.name
u'/SubGroup/AnotherGroup'
```

By the way, you don't have to manually create nested groups one at a time. If you supply a full path, HDF5 will create all the intermediate groups for you:

```
>>> out = f.create_group('/some/big/path')
>>> out
<HDF5 group "/some/big/path" (0 members)>
```

The same goes for creating datasets; just supply the full path you want and HDF5 will fill in the missing pieces.

Group Basics

If you remember nothing else from this chapter, remember this: *groups work mostly like dictionaries*. There are a couple of holes in this abstraction, but on the whole it works surprisingly well. Groups are iterable, and have a subset of the normal Python dictionary API.

Let's add another few objects to our file for the examples that follow:

```
>>> f["Dataset1"] = 1.0
>>> f["Dataset2"] = 2.0
>>> f["Dataset3"] = 3.0
>>> subgroup["Dataset4"] = 4.0
```

Dictionary-Style Access

You got a hint of this dictionary-like behavior from the syntax `group[name] = object`. Objects can be retrieved from a group by name:

```
>>> dset1 = f["Dataset1"]
```

Unlike normal Python dictionaries, you can also use POSIX-style paths to directly access objects in subgroups, without having to tediously open all the groups between here and there:

```
>>> dset4 = f["SubGroup/Dataset4"] # Right
>>> dset4 = f["SubGroup"]["Dataset4"] # Works, but inefficient
```

Attempting to access an empty group raises `KeyError`, although one irritating thing about `h5py` is that you don't get the name of the missing object in the exception:

```
>>> f['BadName']
KeyError: "unable to open object (Symbol table: Can't open object)"
```

There's also the familiar `get` method, which is handy if you don't want to raise an exception:

```
>>> out = f.get("BadName")
>>> print out
None
```

You can take the length of a group—note that this measures the number of objects *directly* attached to the group rather than all the objects in nested subgroups:

```
>>> len(f)
5
>>> len(f["SubGroup"])
2
```

Pythonic iteration is also supported using the familiar `iteritems()` and friends (see “Iteration and Containership” on page 65).

Special Properties

There are a few widgets attached to groups (and datasets) that are very useful when working with the hierarchy in a file.

First is the `.file` property. Attached to every object, this gives you a handy way to retrieve a `File` object for the file in which your object resides:

```
>>> f = h5py.File('propdemo.hdf5', 'w')
>>> grp = f.create_group('hello')
>>> grp.file == f
True
```

This is great when you want to check whether a file is read/write, or just get the filename.

Second is the `.parent` property. This returns the `Group` object that contains your object:

```
>>> grp.parent
<HDF5 group "/" (1 members)>
```

With these two properties, you can avoid most of the path-formatting headaches associated with filesystem work.

Working with Links

What does it mean to give an object a name in the file? From the preceding examples, you might think that the name is part of the object, in the same way that the `dtype` or `shape` are part of a dataset.

But this isn't the case. There's a layer between the group object and the objects that are its members. The two are related by the concept of *links*.

Hard Links

Links in HDF5 are handled in much the same way as in modern filesystems. Objects like datasets and groups don't have an intrinsic name; rather, they have an *address* (byte offset) in the file that HDF5 has to look up. When you assign an object to a name in a

group, that address is recorded in the group and associated with the name you provided to form a *link*.

Among other things, this means that objects in an HDF5 file can have *more than one name*; in fact, they have as many names as there exist links pointing to them. The number of links that point to an object is recorded, and when no more links exist, the space used for the object is freed.

This kind of a link, the default in HDF5, is called a *hard link* to differentiate it from other kinds of links discussed later in this chapter.

Here's an example of the multiple-name behavior. We'll create a simple file containing a group, and create a hard link to it at /x:

```
>>> f = h5py.File('linksdemo.hdf5', 'w')
>>> grp_x = f.create_group('x')
>>> grp_x.name
u'/x'
```

Now we'll create a second link pointing to the group. You can do this using standard Python dictionary-style item assignment:

```
>>> f['y'] = grp_x
```

When we retrieve an object from location /y, we get the same group back:

```
>>> grp_y = f['y']
>>> grp_y == grp_x
True
```

You might wonder what happens to the `.name` property if objects in a file don't have a unique name. Let's see:

```
>>> grp_x.name
u'/x'
>>> grp_y.name
u'/y'
```

HDF5 makes a best effort to return the name used to retrieve an object. But there's no guarantee. If the object has a name, then you'll generally get one when accessing `.name`, but it may not be the one you expect.

What does that mean, "if the object has a name"? It's perfectly legal in HDF5 to create an object *without* a name; just supply `None`:

```
>>> grp_z = f.create_group(None)
>>> print grp_z.name
None
```

The group `grp_z` in this case exists in the file, but there's no way to get there from the root group. If we were to get rid of the Python object `grp_z`, the group would be deleted

and the space in the file reclaimed. To avoid this we can simply link the group into the file structure at our leisure:

```
>>> f['z'] = grpz
>>> grpz.name
u'/z'
```

The multiple names issue also affects the behavior of the `.parent` property. To address this, in `h5py`, `obj.parent` is defined to be the “parent” object according to `obj.name`. For example, if `obj.name` is `/foo/bar`, `obj.parent.name` will be `/foo`.

One way to express this is with the `posixpath` package built into Python:

```
>>> import posixpath
>>> parent = obj.file[posixpath.dirname(obj.name)]
```

To remove links, we use the dictionary-style syntax `del group[name]`:

```
>>> del f['y']
```

Once all hard links to an object are gone (and the object isn’t open somewhere in Python), it’s destroyed:

```
>>> del f['x'] # Last hard link; the group is deleted in the file
```

Free Space and Repacking

When an object (for example, a large dataset) is deleted, the space it occupied on disk is reused for new objects like groups and datasets. However, at the time of writing, HDF5 does not track such “free space” across file open/close cycles. So if you don’t end up reusing the space by the time you close the file, you may end up with a “hole” of unusable space in the file that can’t be reclaimed.

This issue is a high development priority for the HDF Group. In the meantime, if your files seem unusually large you can “repack” them with the `h5repack` tool, which ships with HDF5:

```
$ h5repack bigfile.hdf5 out.hdf5
```

Soft Links

Those of you who have used Linux or Mac OS X will be familiar with “soft” links. Unlike “hard” links, which associate a link name with a particular object in the file, soft links instead store the *path* to an object.

Here’s an example. Let’s create a file and populate it with a single group containing a dataset:

```
>>> f = h5py.File('test.hdf5', 'w')
>>> grp = f.create_group('mygroup')
>>> dset = grp.create_dataset('dataset', (100,))
```


If we were to create a hard link in the root group to the dataset, it would always point to that particular object, even if the dataset were moved or unlinked from mygroup:

```
>>> f['hardlink'] = dset
>>> f['hardlink'] == grp['dataset']
True
>>> grp.move('dataset', 'new_dataset_name')
>>> f['hardlink'] == grp['new_dataset_name']
True
```

Let's move the dataset back, and then create a *soft* link that points to the path /mygroup/dataset. To tell HDF5 that we want to create a soft link, assign an instance of the class `h5py.SoftLink` to a name in the file:

```
>>> grp.move('new_dataset_name', 'dataset')
>>> f['softlink'] = h5py.SoftLink('/mygroup/dataset')
>>> f['softlink'] == grp['dataset']
True
```

`SoftLink` objects are very simple; they only have one property, `.path`, holding the path provided when they are created:

```
>>> softlink = h5py.SoftLink('/some/path')
>>> softlink
<SoftLink to "/some/path">
>>> softlink.path
'/some/path'
```

Keep in mind that instances of `h5py.SoftLink` are purely a Python-side convenience, not a wrapper around anything in HDF5. Nothing happens until you assign one of them to a name in the file.

Returning to our example, since only the path is stored, if we move the dataset and replace it with something else, /softlink would then point to the new object:

```
>>> grp.move('dataset', 'new_dataset_name')
>>> dset2 = grp.create_dataset('dataset', (50,))
>>> f['softlink'] == dset
False
>>> f['softlink'] == dset2
True
```

Soft links are therefore a great way to refer to “the object which resides at /some/particular/path,” rather than any specific object in the file. This can be very handy if, for example, a particular dataset represents some information that needs to be updated without breaking all the links to it elsewhere in the file.

The value of a soft link is *not* checked when it's created. If you supply an invalid path (or the object is later moved/deleted), accessing will fail with an exception. Because of how HDF5 reports such an error, you will get the same exception as you would when trying to access a nonexistent name in the group, `KeyError`:

```
>>> f['broken'] = h5py.SoftLink('/some/nonexistent/object')
>>> f['broken']
KeyError: "unable to open object"
```

By the way, since soft links only record a path, they don't participate in the reference counting that hard links do. So if you have a soft link `/softlink` pointing at an object hard-linked at `/a`, if you delete the object (`del f["/a"]`) it will be destroyed and the soft link will simply break.



You may be wondering what happens when a broken soft link appears in `items()` or `values()`. The answer is that object `None` is used as the value instead.

External Links

Starting with HDF5 1.8, there's an additional type of link in addition to the file-local hard and soft links. *External links* allow you to refer to objects in other files. They're one of the coolest features of HDF5, but one of the most troublesome to keep track of because of their transparency.

An external link has two components: the name of a file, and then the (absolute) name of an object within that file. Like soft links, you create them with a “marker” object, in this case an instance of `h5py.ExternalLink`.

Let's create a file with a single object inside, and then link to it from another file:

```
>>> with h5py.File('file_with_resource.hdf5', 'w') as f1:
...     f1.create_group('mygroup')

>>> f2 = h5py.File('linking_file.hdf5', 'w')
>>> f2['linkname'] = h5py.ExternalLink('file_with_resource.hdf5', 'mygroup')
```

Like soft links, external links are transparent, in the sense that if they're not broken, we get back the group or dataset they point to instead of some intermediate object. So if we access the link we just created, we get a group back:

```
>>> grp = f2['linkname']
>>> grp.name
u'/mygroup'
```

However, if we look more closely we discover that this object resides in a different file:

```
>>> grp.file
<HDF5 file "file_with_resource.hdf5" (mode r+)>
>>> f2
<HDF5 file "linking_file.hdf5" (mode r+)>
```

Keep in mind that this can lead to odd-looking consequences. For example, when you use the `.parent` property on the retrieved object, it points to the root group of the external file, not the file in which the link resides:

```
>>> f2['/linkname'].parent == f2['/']  
False
```

Both the file and object names are checked when you create the external link. So if HDF5 can't find the file, or the specified object within the file, you'll get an exception:

```
>>> f2['anotherlink'] = h5py.ExternalLink('missing.hdf5', '/')  
ValueError: unable to create link (Links: Unable to initialize object)
```

The two main hazards when dealing with external links are (1) that the file the link points to won't be around when it's accessed, and (2) by simply traversing the links in the file, you can “wander” into a different file.

There's not too much we can do about (1); it's up to you to keep files organized and be mindful of what links to what. Hazard (2) is a little more dangerous, particularly since all the “Pythonic” methods of accessing group members, like `iteration`, `items()`, and so on, will include external links. If it's undesirable for your application to cross file boundaries, be sure to check the `.file` property to see where the objects actually reside.

At the moment, there's no way to set a “search path” from `h5py`. When an external link is encountered, HDF5 will first look for the destination file in the same directory as the file with the link, and then in the current working directory. That's it.

A Note on Object Names

You might have noticed that when you retrieve the name of an object, it comes out as a Python Unicode object:

```
>>> f['/foo'].name  
u'/foo'
```

This is intentional, and following the improved Unicode support of HDF5 1.8 (and Python 3) it was introduced in `h5py` version 2.0. Object names in the file are always treated as “text” strings, which means they represent sequences of characters. In contrast, “byte” strings are sequences of 8-bit numbers that can often but not always store ASCII or Latin-1 encoded text.

The great thing about this is that international characters are supported for all object names in the file; you don't have to “ASCII-ize” anything to fit it into the HDF5 system. Names are stored using the most-compatible storage strategy possible for maximum compatibility with older versions (1.6) of HDF5.

To take advantage of this, simply supply a Unicode string when creating an object or making a new link:

```
>>> grp = f.create_group('e_with_accent_\u00E9')
>>> print grp.name
/e_with_accent_é
```

On the backend, h5py converts your string to the HDF5-approved UTF-8 encoding before storing it. When you supply a “regular” or “byte” string (as in most of the previous examples), h5py uses your string as is. It’s technically possible to store non-UTF-8 strings like this, although such use is strongly discouraged. If you do happen to receive a file with such “noncompliant” object names, h5py will simply pass back the raw byte string and let you figure it out.

Using get to Determine Object Types

We mentioned that the familiar dictionary-style method `get` was also available on `Group` objects, and showed how to handle missing group members without raising `KeyError`. But this version of `get` is a little more capable than the Python `get`.

There are two additional keywords in addition to the dictionary-style default: `getclass` and `getlink`. The `getclass` keyword lets you retrieve the *type* of an object without actually having to open it. At the HDF5 level, this only requires reading some metadata and is consequently very fast.

Here’s an example: first we’ll create a file containing a single group and a single dataset:

```
>>> f = h5py.File('get_demo.hdf5', 'w')
>>> f.create_group('subgroup')
>>> f.create_dataset('dataset', (100,))
```

Using `get`, the *type* of object can be retrieved:

```
>>> for name in f:
...     print name, f.get(name, getclass=True)
dataset <class 'h5py._hl.dataset.Dataset'>
subgroup <class 'h5py._hl.group.Group'>
```

The second keyword, `getlink`, lets you determine the properties of the link involved:

```
>>> f['softlink'] = h5py.SoftLink('/subgroup')
>>> with h5py.File('get_demo_ext.hdf5', 'w') as f2:
...     f2.create_group('egroup')
>>> f['extlink'] = h5py.ExternalLink('get_demo_ext.hdf5', '/egroup')

>>> for name in f:
...     print name, f.get(name, getlink=True)
dataset <h5py._hl.group.HardLink object at 0x047277F0>
extlink <ExternalLink to "/egroup" in file "get_demo_ext.hdf5">
softlink <SoftLink to "/subgroup">
subgroup <h5py._hl.group.HardLink object at 0x047273B0>
```

You'll notice that instances of `SoftLink` and `ExternalLink` were returned, complete with path information. This is the official way to retrieve such information after the link is created.

For the hard links at `subgroup` and `dataset`, there's also an instance of something called `h5py.HardLink`. This exists solely to support the use of `get`; it has no other function and no properties or methods.

Finally, if all you care about is the *kind* of link involved, and not the exact values of the paths and files involved, you can combine the `getclass` and `getlink` keywords to return the link class:

```
>>> for name in f:
...     print name, f.get(name, getclass=True, getlink=True)
dataset <class 'h5py._hl.group.HardLink'>
extlink <class 'h5py._hl.group.ExternalLink'>
softlink <class 'h5py._hl.group.SoftLink'>
subgroup <class 'h5py._hl.group.HardLink'>
```



For many of the classes involved here, you may notice that they were originally defined in the subpackage `h5py._hl`, for example `h5py._hl.group.SoftLink` shown earlier. This is an implementation detail that may change; when doing `isinstance` checks, etc., use the names directly attached to the `h5py` package (e.g., `h5py.SoftLink`).

Using require to Simplify Your Application

Unlike Python dictionaries, you can't directly overwrite the members of a group:

```
>>> f = h5py.File('require_demo.hdf5', 'w')
>>> f.create_group('x')
>>> f.create_group('y')
>>> f.create_group('y')
ValueError: unable to create group (Symbol table: Unable to initialize object)
```

This also holds true for manually hard-linking objects:

```
>>> f['y'] = f['x']
ValueError: unable to create link (Links: Unable to initialize object)
```

This is an intentional feature designed to prevent data loss. Since objects are immediately deleted when you unlink them from a group, you have to explicitly delete the link rather than having HDF5 do it for you:

```
>>> del f['y']
>>> f['y'] = f['x']
```

This leads to some headaches in real-world code. For example, a fragment of analysis code might create a file and write the results to a dataset:

```
>>> data = do_large_calculation()
>>> with h5py.File('output.hdf5') as f:
...     f.create_dataset('results', data=data)
```

If there are many datasets and groups in the file, it might not be appropriate to overwrite the entire file every time the code runs. But if we don't open in `w` mode, then our program will only work the first time, unless we manually remove the output file every time it runs.

To deal with this, `create_group` and `create_dataset` have companion methods called `require_group` and `require_dataset`. They do exactly the same thing, only first they check for an existing group or dataset and return it instead.

Both versions take exactly the same arguments and keywords. In the case of `require_dataset`, `h5py` also checks the requested shape and dtype against any existing dataset and fails if they don't match:

```
>>> f.create_dataset('dataset', (100,), dtype='i')
>>> f.require_dataset('dataset', (100,), dtype='f')
TypeError: Datatypes cannot be safely cast (existing int32 vs new f)
```

There's a minor detail here, in that a conflict is only deemed to occur if the shapes don't match, or the *requested precision* of the datatype is *higher* than the existing precision. So if there's a preexisting `int64` dataset, then `require_dataset` will succeed if `int32` is requested:

```
>>> f.create_dataset('int_dataset', (100,), dtype='int64')
>>> f.require_dataset('int_dataset', (100,), dtype='int32')
```

The NumPy casting rules are used to check for conflicts; you can test the types yourself using `np.can_cast`.

Iteration and Containership

Iteration is a core Python concept, key to writing “Pythonic” code that runs quickly and that your colleagues can understand. It's also a natural way to explore the contents of groups.

How Groups Are Actually Stored

In the HDF5 file, group members are indexed using a structure called a “B-tree.” This isn't a computer science text, so we won't spend too long on the subject, but it's valuable to have a rough understanding of what's going on behind the scenes, especially if you're dealing with groups that have thousands or hundreds of thousands of items.

“B-trees” are data structures that are great for keeping track of large numbers of items, while still making retrieval (and addition) of items fast. They work by taking a collection

of items, each of which is orderable according to some scheme like a string name or numeric identifier, and building up a tree-like “index” to rapidly retrieve an item.

For example, if you have an HDF5 group with a single member, and another group with a million members, it *doesn't* take a million times as long to open an object in the latter group. Group members are indexed by name, so if you know the name of an object then HDF5 can traverse the index and quickly retrieve the item. The same is true when creating a new group member; HDF5 doesn't have to “insert” the member into the middle of a big table somewhere, shuffling all the entries around.

Of course, all of this is transparent to the user. Every group in an HDF5 file comes with an index that tracks members in alphabetical order. Keep in mind this means “C-style” alphabetical order (whimsically called “ASCIIbetical” order):

```
>>> f = h5py.File('iterationdemo.hdf5', 'w')
>>> f.create_group('1')
>>> f.create_group('2')
>>> f.create_group('10')
>>> f.create_dataset('data', (100,))
>>> f.keys()
[u'1', u'10', u'2', u'data']
```

Files can also contain other optional indices, for example those that track object creation time, but h5py doesn't expose them.

This brings us to the first point: h5py will generally iterate over objects in the file in alphabetical order (especially for small groups), but you shouldn't rely on this behavior. Behind the scenes, HDF5 is actually retrieving objects in so-called *native* order, which basically means “as fast as possible.” The only thing that's guaranteed is that if you don't modify the group, the order will remain the same.

Dictionary-Style Iteration

In keeping with the general convention that *groups work like dictionaries*, iterating over a group in HDF5 provides the names of the members. Remember, these will be supplied as Unicode strings:

```
>>> [x for x in f]
[u'1', u'10', u'2', u'data']
```

There are also `iterkeys` (equivalent to the preceding), `itervalues`, and `iteritems` methods, which do just what you'd expect:

```
>>> [y for y in f.itervalues()]
[<HDF5 group "/"1" (0 members)>,
 <HDF5 group "/"10" (0 members)>,
 <HDF5 group "/"2" (0 members)>,
 <HDF5 dataset "data": shape (100,), type "<f4">]
```

```
>>> [(x,y) for x, y in f.iteritems()]
[(u'1', <HDF5 group "/1" (0 members)>),
 (u'10', <HDF5 group "/10" (0 members)>),
 (u'2', <HDF5 group "/2" (0 members)>),
 (u'data', <HDF5 dataset "data": shape (100,), type "<f4">)]
```

There are also the standard `keys`, `items`, and `values` methods, which produce lists equivalent to the three preceding examples. This brings us to the first performance tip involving iteration and groups: unless you *really* want to produce a list of the 10,000 objects in your group, use the `iter*` methods.



If you're using Python 3, you'll notice that you have only the `keys`, `values`, and `items` methods. That's OK; like dictionaries, under Python 3 these return iterables, not lists.

Containership Testing

This is another seemingly obvious performance issue that crops up from time to time. If you're writing code like this, DON'T:

```
>>> if 'name' in group.keys():
```

This creates and throws away a list of all your group members every time you use it. By instead using the standard Python containership test, you can leverage the underlying HDF5 index on object names, which will go very, very fast:

```
>>> if 'name' in group:
```

Critically, you can also use paths spanning several groups, although it's very slightly slower since the intermediate groups have to be inspected by HDF5:

```
>>> if 'some/big/path' in group:
```

Very handy. Keep in mind that like accessing group members, the POSIX-style “parent directory” symbol `..` *won't work*. You won't even get an error message; HDF5 will look for a group named `..` and determine it's not present:

```
>>> '..' in f['/1']
False
```

If you're manipulating POSIX-style strings and run into this problem, consider “normalizing” your paths using the `posixpath` package:

```
>>> grp = f['/1']
>>> path = "../1"

>>> import posixpath as pp
>>> path = pp.normpath(pp.join(grp.name, path))
>>> path
u'/1'
```



```
>>> path in grp
True
```

Multilevel Iteration with the Visitor Pattern

Basic iteration works fine for the contents of a single group. But what if you want to iterate over every single object in the file? Or all objects “below” a certain group?

In the HDF5 world, this is accomplished by *visitor* iteration. Rather than HDF5 supplying you with an iterable, you provide a *callable* and HDF5 calls it with an argument or two for every object.

Visit by Name

Your entry point is the `visit` method on the `Group` class. Let’s create a simple file to test it out:

```
>>> f = h5py.File('visit_test.hdf5', 'w')

>>> f.create_dataset('top_dataset', data=1.0)

>>> f.create_group('top_group_1')
>>> f.create_group('top_group_1/subgroup_1')
>>> f.create_dataset('top_group_1/subgroup_1/sub_dataset_1', data=1.0)

>>> f.create_group('top_group_2')
>>> f.create_dataset('top_group_2/sub_dataset_2', data=1.0)
```

We can supply any callable to `visit`, which takes one argument, the object name:

```
>>> def printname(name):
...     print name

>>> f.visit(printname)
top_dataset
top_group_1
top_group_1/subgroup_1
top_group_1/subgroup_1/sub_dataset_1
top_group_2
top_group_2/sub_dataset_2
```

No particular order is guaranteed, except that when `visit` enters a subgroup, all the members will be visited before moving on to the next subgroup. For example, everything under `top_group_1` is listed together, and so is everything under `top_group_2`.

You’re not required to visit the entire file; `visit` works just fine on subgroups:

```
>>> grp = f['top_group_1']
>>> grp.visit(printname)
subgroup_1
subgroup_1/sub_dataset_1
```

The *visitor* pattern is a little different from standard Python iteration, but is quite powerful once you get used to it. For example, here's a simple way to get a list of every single object in the file:

```
>>> mylist = []
>>> f.visit(mylist.append)
```



As with all object names in the file, the names supplied to `visit` are “text” strings (unicode on Python 2, `str` on Python 3). Keep this in mind when writing your callbacks.

Multiple Links and visit

Of course, we know that an HDF5 file is not just a simple tree. Hard links are a great way to share objects between groups. But how do they interact with `visit`?

Let's add a hard link to the subgroup we just explored (`top_group_1`), and run `visit` again to see what happens:

```
>>> grp['hardlink'] = f['top_group_2']
>>> grp.visit(printname)
hardlink
hardlink/sub_dataset_2
subgroup_1
subgroup_1/sub_dataset_1
```

Not bad. The group at `/top_group_2` is effectively “mounted” in the file at `/top_group_1/hardlink`, and `visit` explores it correctly.

Now let's try something a little different. We'll undo that last hard link, and try to trick `visit` into visiting `sub_dataset_1` twice:

```
>>> del grp['hardlink']
>>> grp['hardlink_to_dataset'] = grp['subgroup_1/sub_dataset_1']
>>> grp.visit(printname)
hardlink_to_dataset
subgroup_1
```

What happened? We didn't see `sub_dataset_1` in the output this time.

By design, each object in a file will be visited only *once*, regardless of how many links exist to the object. Among other things, this eliminates the possibility of getting stuck in an endless loop, as might happen if some clever person were to try the following:

```
>>> f['/root'] = f['/']
```

There is a trade-off. As we saw in our initial discussion of hard links, there's no such thing as the “original” or “real” name for an object. So if multiple links point to your dataset, when `visit` supplies a name it may not be the one you expect.

Visiting Items

Given the name supplied to your callback, you could retrieve the object by simply using `getitem` on the group you're iterating over:

```
>>> def printobj(name):
...     print grp[name]
```

But that's a pain; since the `name` argument supplied by `visit` is a *relative* path, your function has to know in advance what group it'll be applied to. The previous example will work properly only when applied to `grp`.

Thankfully, HDF5 provides a more general way to deal with this. The method `visititems` supplies both the relative name and an instance of each object:

```
>>> def printobj2(name, obj):
...     print name, obj

>>> grp.visititems(printobj2)
hardlink_to_dataset <HDF5 dataset "hardlink_to_dataset": shape (), type "<f8">
subgroup_1 <HDF5 group "/top_group_1/subgroup_1" (1 members)>
```

Since each object has to be opened, there is some overhead involved. You're better off using `visititems` only in the case where you really need access to each object; for example, if you need to inspect attributes.

One way to make `visit` a little more generic is by using the built-in Python widget `functools.partial`. For example, here's a trivial function that prints the absolute path of each object in the group:

```
>>> import posixpath
>>> from functools import partial

>>> def print_abspath(somegroup, name):
...     """ Print *name* as an absolute path
...         somegroup: HDF5 base group (*name* is relative to this)
...         name:      Object name relative to *somegroup*
...     """
...     print posixpath.join(somegroup.name, name)

>>> grp.visit(partial(print_abspath, grp))
/top_group_1/hardlink_to_dataset
/top_group_1/subgroup_1
```

Using this technique, you can avoid “embedding” the group you intend to iterate over in the function itself.

Canceled Iteration: A Simple Search Mechanism

There's a simple way to “bail out” when visiting items. You might notice that our `print name` function has no explicit return value; in Python that means that the function re-

turns None. If you return anything else, the visit or visititems method will immediately stop *and return that value*.

Let's suppose that we want to find a dataset that has an attribute with a particular value:

```
>>> f['top_group_2/sub_dataset_2'].attrs['special'] = 42
```

Here's a function that will find such an object, when supplied to visititems:

```
>>> def findspecial(name, obj):
...     if obj.attrs.get('special') == 42:
...         return obj

>>> out = f.visititems(findspecial)
>>> out
<HDF5 dataset "sub_dataset_2": shape (), type "<f8">
```

Copying Objects

HDF5 includes built-in facilities for copying objects from one place to another, freeing you from the tedious job of recursively walking the HDF5 tree, checking for duplicate links, copying over attributes, etc.

Single-File Copying

Let's create a simple file to test this, with two groups and a dataset:

```
>>> f = h5py.File('copytest', 'w')
>>> f.create_group('mygroup')
>>> f.create_group('mygroup/subgroup')
>>> f.create_dataset('mygroup/apples', (100,))
```

Copying a dataset is straightforward, and results in a brand-new dataset, not a reference or link to the old one:

```
>>> f.copy('/mygroup/apples', '/oranges')
>>> f['oranges'] == f['mygroup/apples']
False
```

The great thing about the built-in HDF5 copy() is that it correctly handles recursively copying groups:

```
>>> f.copy('mygroup', 'mygroup2')
>>> f.visit(printname)
oranges
mygroup
mygroup/apples
mygroup/subgroup
mygroup2
mygroup2/apples
mygroup2/subgroup
```

You're not limited to using paths for source and destination. If you already have an open `Dataset` object, for example, you can copy it to a `Group` or `File` object:

```
>>> dset = f['/mygroup/apples']
>>> f.copy(dset, f)
>>> f.visit(printname)
apples
oranges
mygroup
mygroup/dataset
mygroup/subgroup
mygroup2
mygroup2/dataset
mygroup2/subgroup
```

Since the destination is a group, the dataset is created with its “base name” of `apples`, analogous to how files are moved with the UNIX `cp` command.

There's no requirement that the source and destination be the same file. This is one of the advantages of using `File` or `Group` objects instead of paths; the corresponding objects will be copied regardless of which file they reside in. If you're trying to write generic code, it's good to keep this in mind.

Object Comparison and Hashing

Let's take a break from links and iteration to discuss a more subtle aspect of how HDF5 behaves. In lots of the preceding examples, we used Python's equality operator to see if two groups are “the same thing”:

```
>>> f = h5py.File('objectdemo.hdf5', 'w')
>>> grpx = f.create_group('x')
>>> grpy = f.create_group('y')
>>> grpx == f['x']
True
>>> grpx == grpy
False
```

If we investigate further, we discover that this kind of equality testing is independent of whether the Python objects are one and the same:

```
>>> id(grpx) # Uniquely identifies the Python object "grpx"
73399280
>>> id(f['x'])
73966416
```

In `h5py`, equality testing uses the low-level HDF5 facilities to determine which references (*identifiers*, in the HDF5 lingo) point to the same groups or datasets on disk. This information is also used to compute the hash of an object, which means you can safely use `Group`, `File`, and `Dataset` objects as dictionary keys or as the members of sets:

```
>>> hash(grpx)
587327447
>>> hash(f['x'])
587327447
```

There's one more wrinkle for equality testing, and you may bump into it when using the `.file` property on objects: `File` and `Group` instances will compare equally when the `Group` instance represents the root group:

```
>>> f == f['/']
True
```

This is a consequence of the “double duty” `File` instances perform, representing both your file on disk and also the root group on the HDF5 side.

Finally, the truth value of an HDF5 object lets you see whether it's alive or dead:

```
>>> bool(grpx)
True
>>> f.close()
>>> grpx
<Closed HDF5 group>
>>> bool(grpx)
False
```

Next, we discuss one thing that makes HDF5 so useful in real-world science: the ability to store data and metadata together side by side using attributes.

Storing Metadata with Attributes

Groups and datasets are great for keeping data organized in a file. But the feature that really turns HDF5 into a scientific database, instead of just a file format, is *attributes*.

Attributes are pieces of metadata you can stick on objects in the file. They can hold equipment settings, timestamps, computation results, version numbers, virtually anything you want. They're a key mechanism for making *self-describing* files. Unlike simple binary formats that just hold arrays of numbers, judicious use of metadata makes your files scientifically useful all on their own.

Attribute Basics

You can attach attributes to any kind of object that is linked into the HDF5 tree structure: groups, datasets, and even named datatypes. To demonstrate, let's create a new file containing a single dataset:

```
>>> f = h5py.File('attrsdemo.hdf5', 'w')
>>> dset = f.create_dataset('dataset', (100,))
```

Looking at the properties attached to the `dset` object, there's one called `.attrs`:

```
>>> dset.attrs
<Attributes of HDF5 object at 73767504>
```

This is a little proxy object (an instance of `h5py.AttributeManager`) that lets you interact with attributes in a Pythonic way. As was the case with groups, the main thing to keep in mind here is that the `attrs` object works mostly like a Python dictionary.

For example, you can create a new attribute simply by assigning a name to a value:

```
>>> dset.attrs['title'] = "Dataset from third round of experiments"
>>> dset.attrs['sample_rate'] = 100e6      # 100 MHz digitizer setting
>>> dset.attrs['run_id'] = 144
```


When retrieving elements, we get back the actual value, not an intermediate object like a Dataset:

```
>>> dset.attrs['title']
'Dataset from third round of experiments'
>>> dset.attrs['sample_rate']
100000000.0
>>> dset.attrs['run_id']
144
```

Like groups (and Python dictionaries), iterating over the `.attrs` object provides the attribute names:

```
>>> [x for x in dset.attrs]
[u'title', u'sample_rate', u'run_id']
```

You'll notice that like object names, the names of attributes are always returned as “text” strings; this means unicode on Python 2, which explains the `u` prefix.

Attributes don't have the same strict rules as groups for item deletion. You can freely overwrite attributes just by reusing the name:

```
>>> dset.attrs['another_id'] = 42
>>> dset.attrs['another_id'] = 100
```

Trying to access missing attributes raises `KeyError`, although as with Group you don't get the name of the missing attribute:

```
>>> del dset.attrs['another_id']
>>> dset.attrs['another_id']
KeyError: "can't open attribute (Attribute: Can't open object)"
```

There are also the usual methods like `iterkeys`, `iteritems`, `values`, and so on. They all do what you expect:

```
>>> [(name, val) for name, val in dset.attrs.iteritems()]
[(u'title', 'Dataset from third round of experiments'),
 (u'sample_rate', 100000000.0),
 (u'run_id', 144)]
```

There generally aren't that many attributes attached to an object, so worrying about items versus `iteritems`, etc., is less important from a performance perspective.

There is also a `get` method that (unlike the Group version) is a dictionary-style get:

```
>>> dset.attrs.get('run_id')
144
>>> print dset.attrs.get('missing')
None
```

Type Guessing

When you create a dataset, you generally specify the data type you want by providing a NumPy dtype object. There are exceptions; for example, you can get a single-precision float by omitting the dtype when calling `create_dataset`. But every dataset has an explicit dtype, and you can always discover what it is via the `.dtype` property:

```
>>> dset.dtype
dtype('float32')
```

In contrast, with attributes h5py generally hides the type from you. It's important to remember that there *is* a definite type in the HDF5 file. The dictionary-style interface to attributes just means that it's usually inferred from what you provide.

Let's flush our file to disk with:

```
>>> f.flush()
```

and look at it with `h5ls`:

```
$ h5ls -vlr attrsdemo.hdf5
Opened "attrsdemo.hdf5" with sec2 driver.
/
  Group
    Location: 1:96
    Links: 1
  /dataset
    Dataset {100/100}
      Attribute: run_id scalar
        Type: native int
        Data: 144
      Attribute: sample_rate scalar
        Type: native double
        Data: 1e+08
      Attribute: title scalar
        Type: variable-length null-terminated ASCII string
        Data: "Dataset from third round of experiments"
    Location: 1:800
    Links: 1
    Storage: 400 logical bytes, 0 allocated bytes
    Type: native float
```

In most cases, the type is determined by simply passing the value to `np.array` and then storing the resulting object. For integers on 32-bit systems you would get a 32-bit (“native”) integer:

```
>>> np.array(144).dtype
dtype('int32')
```

This explains the “native int” type for `run_id`.

You're not limited to scalar values, by the way. There's no problem storing whole NumPy arrays in the file:

string as an attribute. That created a *variable-length* ASCII string (“[Variable-Length Strings](#)” on page 89).

In contrast, an instance of `np.string_` would get stored as a *fixed-length* string in the file:

```
>>> dset.attrs['title_fixed'] = np.string_("Another title")
```

This generally isn’t an issue, but some older FORTRAN-based programs can’t deal with variable-length strings. If this is a problem for your application, use `np.string_`, or equivalently, arrays of NumPy type `S`.

By the way, you can also store Unicode strings in the file. They’re written out with the HDF5-approved UTF-8 encoding:

```
>>> dset.attrs['Yet another title'] = u'String with accent (\u00E9)'
>>> f.flush()
```

Here’s what the file looks like now, with our fixed-length and Unicode strings inside:

```
$ h5ls -vlr attrsdemo.hdf5/dataset
Opened "attrsdemo.hdf5" with sec2 driver.
dataset                Dataset {100/100}
  Attribute: Yet\ another\ title scalar
    Type:      variable-length null-terminated UTF-8 string
    Data:      "String with accent (\3777777703\3777777651)"
  Attribute: ones      scalar
    Type:      object reference
    Data:      DATASET-1:70568
  Attribute: run_id    scalar
    Type:      native int
    Data:      144
  Attribute: sample_rate scalar
    Type:      native double
    Data:      1e+08
  Attribute: title     scalar
    Type:      variable-length null-terminated ASCII string
    Data:      "Dataset from third round of experiments"
  Attribute: title_fixed scalar
    Type:      13-byte null-padded ASCII string
    Data:      "Another title"
Location: 1:800
Links: 1
Storage: 400 logical bytes, 0 allocated bytes
Type: native float
```

There is one more thing to mention about strings, and it has to do with the strict separation in Python 3 between *byte* strings and *text* strings.

When you read an attribute from a file, you generally get an object with the same type as in HDF5. So if we were to store a NumPy `int32`, we would get an `int32` back.

In Python 3, this means that most of the HDF5 strings “in the wild” would be read as *byte* strings, which are very awkward to deal with. So in Python 3, scalar strings are always converted to text strings (type `str`) when they are read.

Python Objects

The question of storing generic Python objects in HDF5 comes up now and then. You’ll notice that you can’t store an arbitrary object as an attribute (or indeed, as a dataset) in HDF5:

```
>>> dset.attrs['object'] = {}
TypeError: Object dtype dtype('object') has no native HDF5 equivalent
```

This is intentional. As the error message suggests, HDF5 has no “native,” built-in type to represent a Python object, and serialized objects in a portability-oriented format like HDF5 are generally recognized as bad news. Storing data in “blob” form defeats the wonderful type system and interoperability of HDF5.

However, I can’t tell you how to write your application. If you really want to store Python objects, the best way to do so is by “pickling” (serializing) them to a string:

```
>>> import pickle
>>> pickled_object = pickle.dumps({'key': 42}, protocol=0)
>>> pickled_object
"(dp0\nS'key'\npi\nI42\ns."
>>> dset.attrs['object'] = pickled_object
>>> obj = pickle.loads(dset.attrs['object'])
>>> obj
{'key': 42}
```

You will have to manually keep track of which strings are pickled objects. Technically strings created in this fashion support only ASCII characters, so it’s best to stick with pickle protocol “0.”

Explicit Typing

Sometimes, for external compatibility, you may need to create attributes with very precise data types and the default type guessing won’t do. Or, you may have received a file from a colleague and don’t want to change the types of attributes by overwriting them.

There are a couple of mechanisms to deal with this. The `.attrs` proxy object has a method `create`, which takes a name, value, *and* a dtype:

```
>>> f = h5py.File('attrs_create.hdf5', 'w')
>>> dset = f.create_dataset('dataset', (100,))
>>> dset.attrs.create('two_byte_int', 190, dtype='i2')
>>> dset.attrs['two_byte_int']
190
>>> f.flush()
```

Looking at the file in h5ls:

```
$ h5ls -vlr attrs_create.hdf5
Opened "attrs_create.hdf5" with sec2 driver.
/
  Group
    Location: 1:96
    Links: 1
  /dataset
    Dataset {100/100}
    Attribute: two_byte_int scalar
      Type: native short
      Data: 190
    Location: 1:800
    Links: 1
    Storage: 400 logical bytes, 0 allocated bytes
    Type: native float
```

This is a great way to make sure you get the right *flavor* of string. Unlike scalar strings, by default when you provide an array-like object of strings, they get sent through NumPy and end up as fixed-length strings in the file:

```
>>> dset.attrs['strings'] = ["Hello", "Another string"]
>>> dset.attrs['strings']
array(['Hello', 'Another string'],
      dtype='<S14')
```

In contrast, if you specify the “variable-length string” special dtype (see [Chapter 7](#)):

```
>>> dt = h5py.special_dtype(vlen=str)
>>> dset.attrs.create('more_strings', ["Hello", "Another string"], dtype=dt)
>>> dset.attrs['more_strings']
array(['Hello', 'Another string'], dtype=object)
```

Looking at the file, the two attributes have subtly different storage techniques. The original attribute is stored as a pair of 14-byte fixed-length strings, while the other is stored as a pair of variable-length strings:

```
$ h5ls -vlr attrs_create.hdf5
Opened "attrs_create.hdf5" with sec2 driver.
/
  Group
    Location: 1:96
    Links: 1
  /dataset
    Dataset {100/100}
    Attribute: more_strings {2}
      Type: variable-length null-terminated ASCII string
      Data: "Hello", "Another string"
    Attribute: strings {2}
      Type: 14-byte null-padded ASCII string
      Data: "Hello" '\000' repeats 8 times, "Another string"
    Attribute: two_byte_int scalar
      Type: native short
      Data: 190
    Location: 1:800
    Links: 1
```

```
Storage: 400 logical bytes, 0 allocated bytes
Type:    native float
```

It may seem like a small distinction, but when talking to third-party code this can be the difference between a working program and an error message.

Finally, there's another convenience method called `modify`, which as the name suggests preserves the type of the attribute:

```
>>> dset.attrs.modify('two_byte_int', 33)
>>> dset.attrs['two_byte_int']
33
```

Keep in mind this may have unexpected consequences when the type of the attribute can't hold the value you provide. In this case, the value will clip:

```
>>> dset.attrs.modify('two_byte_int', 40000)
>>> dset.attrs['two_byte_int']
32767
```

Real-World Example: Accelerator Particle Database

Here's an example of how the groups, datasets, and attributes in HDF5 can be combined to solve a real-world data management problem. Recently, the University of Colorado installed a new electrostatic dust accelerator facility under a grant from NASA. This device fires charged micrometer-sized dust grains into a target chamber at speeds ranging from 1–100 km/s, to simulate the impact of dust grains on surfaces and hardware in space.

Application Format on Top of HDF5

The machine generates huge quantities of data. Every particle, and there can be up to 10 per second for hours on end, generates three digitized waveforms 100,000 points long. A computer system analyzes these waveforms to figure out what the mass of the particle is and how fast it's going. The resulting waveforms and speed/mass estimates are recorded in an HDF5 file for use by the project scientists.

So the basic unit is a particle “event,” which has three floating-point waveforms, each of which has some other properties like sampling rate, digitizer range, etc. Then for each event we have metadata estimating particle mass and speed, as well as some top-level metadata like a file timestamp.

Let's use `h5ls` to peek inside one of these files:

```
Opened "November_Run3.hdf5" with sec2 driver.
/
  Group
    Attribute: timestamp scalar
      Type:      native long long
      Data:      1352843341201
```

```

Attribute: version_number scalar
          Type:      native int
          Data:      1
Location:  1:96
Links:     1
/0
          Group
Attribute: experiment_name scalar
          Type:      5-byte null-terminated ASCII string
          Data:      "Run3"
Attribute: id_dust_event scalar
          Type:      native long long
          Data:      210790
Attribute: mass          scalar
          Type:          native float
          Data:          3.81768e-17
Attribute: velocity      scalar
          Type:          native float
          Data:          9646.3
Location:  1:11637136
Links:     1

/0/first_detector      Dataset {100000/100000}
Attribute: dt          scalar
          Type:        native float
          Data:        2e-08
Location:  1:16048056
Links:     1
Storage:   400000 logical bytes, 400000 allocated bytes, 100.00% utilization
Type:      native float

/0/second_detector     Dataset {100000/100000}
Attribute: dt          scalar
          Type:        native float
          Data:        2e-08
Location:  1:16449216
Links:     1
Storage:   400000 logical bytes, 400000 allocated bytes, 100.00% utilization
Type:      native float

/0/third_detector      Dataset {100000/100000}
Attribute: dt          scalar
          Type:        native float
          Data:        2e-08
Location:  1:16449616
Links:     1
Storage:   400000 logical bytes, 400000 allocated bytes, 100.00% utilization
Type:      native float

/1
...
          Group

```


There's a lot going on here, but it's pretty straightforward. The root group has attributes for a timestamp (when the file was written), along with a version number for the “format” used to structure the file using groups, datasets, and attributes.

Then each particle that goes down the beamline has its own group. The group attributes record the analyzed mass and velocity, along with an integer that uniquely identifies the event. Finally, the three waveforms with our original data are recorded in the particle group. They also have an attribute, in this case giving the sampling interval of the time series.

Analyzing the Data

The crucial thing here is that the *metadata* required to make sense of the raw waveforms is stored *right next to the data*. For example, time series like our waveforms are useless unless you also know the time spacing of the samples. In the preceding file, that time interval (*dt*) is stored as an attribute on the waveform dataset. If we wanted to plot a waveform with the correct time scaling, all we have to do is:

```
import pyplot as p
f = h5py.File("November_Run3.hdf5", 'r')

# Retrieve HDF5 dataset
first_detector = f['/0/first_detector']

# Make a properly scaled time axis
x_axis = np.arange(len(first_detector))*first_detector.attrs['dt']

# Plot the result
p.plot(x_axis, first_detector[...])
```

There's another great way HDF5 can simplify your analysis. With other formats, it's common to have an input file or files, a code that processes them, and a “results” file with the output of your computation. With HDF5, you can have *one* file containing both the input data and the results of your analysis.

For example, suppose we wrote a piece of code that determined the electrical charge on the particle from the waveform data. We can store this right in the file next to the estimates for mass and velocity:

```
from some_science_package import charge_estimator

def update_particle_group(grp):

    # Retrieve waveform data
    first_det = grp['first_detector'][...]
    second_det = grp['second_detector'][...]

    # Retrieve time scaling data
    dt = grp['first_detector'].attrs['dt']
```

```

# Perform charge estimation
charge = charge_estimator(first_det, second_det, interval=dt)

# Write charge to file
grp.attrs['charge'] = charge
print "For group %s, got charge %.2g" % (grp.name, charge)

>>> for grp in f.itervalues():
...     update_particle_group(grp)

```

The same goes for analysis that creates output datasets instead of just scalars. The key point here is to transition from thinking of the HDF5 container as a *file* to treating it as a *database*.



Don't get carried away. Keep backups of your data in case you accidentally do something wrong.

We've covered the four main objects in the HDF5 universe: files, groups, datasets, and attributes. Now it's time to take a break and talk about the HDF5 *type* system, and what it can do for you.